

5. Resampling Methods

5.1 Overview

5.2 Validation set approach

5.3 Cross Validation (CV)

5.4 Bootstrap

5.1 Overview

What and why ?

- ▶ *Resampling is an important statistical tool.*
- ▶ *Pretending the data as population and repeatedly draw sample from the data.*
- ▶ *Main **goal**: assess the validity/accuracy of statistical methods and models.*
 - ▶ **Cross-validation**: estimate the test error of models
 - ▶ **Bootstrap**: quantify the uncertainty of estimators

Validation and Cross Validation

- ▶ *Validation set approach*
- ▶ *LOOCV (Leave-one-out cross validation)*
- ▶ *K-fold cross validation (KCV)*

5.1 Overview

Bootstrap

▶ *Sampling with replacement (typically) n times, n is the sample size of data.*

▶ *Especially useful in statistical inference to quantify the uncertainty of estimates (can be even more accurate than normal approximation).*

▶ *Used in ensemble methods of machine learning, for example,*

▶ **Bagging**

▶ **Random forest**

5.1 Overview

Training error is too optimistic !

- ▶ *Training error easily computable with training data.*
- ▶ *Because of possibility of over-fit, it cannot be used to properly assess test error.*
- ▶ *It is possible to "estimate" the test error, by, for example, making adjustments of the training error*
- ▶ *The adjusted R-squared, AIC, BIC, etc serve this purpose.*
- ▶ *These methods rely on certain assumptions and are not for general purpose.*

5.1 Overview

Test error: cross-validation

- ▶ *Test error would be also easily computable, if test data are well designated.*
- ▶ *Shall have to create “test data” for the purpose of computing test error.*
- ▶ *Artificially separate data into “training data” and “test data” for validation purpose is called cross-validation.*
- ▶ *The “test data” here should be more accurately called validation data or hold out data, meaning that they not used in training.*
- ▶ *Model fitting only uses the training data.*

5.1 Overview

Ideal scenario for performance assessment

In a “data-rich” scenario, we can afford to separate the data into three parts:

- ▶ **Training data:** *used to train various models.*
- ▶ **Validation data:** *used to assess the models and identify the best*
- ▶ **Test data:** *test the results of the best model.*

▶ *Usually, people also call validation data or hold-out data as test data.*

5.1 Overview

Difference between validation data and test data

Validation data

This dataset is used during model training to fine-tune the parameters of the model and to prevent overfitting. Typically, the initial training process involves splitting the original data into a training set and a validation set. The training set is used to train the model, while the validation set is used to adjust model parameters, tune hyperparameters, and make decisions about the model architecture. This helps to ensure the model is not just memorizing the training data but is learning to generalize from it. *For example, to tune the neighbour size in KNN.*

Test data

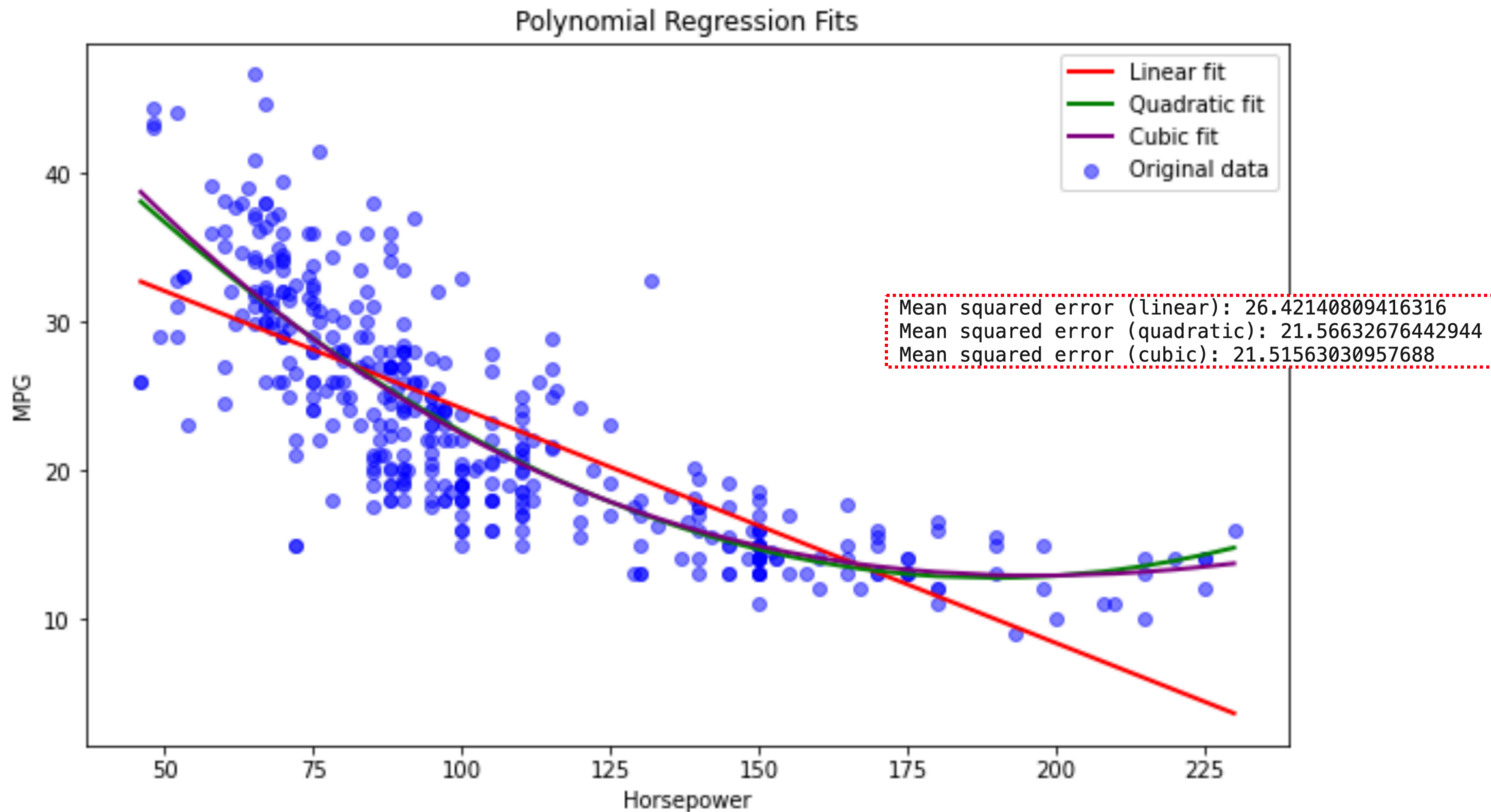
This dataset is used after the model has been trained and validated. It serves as “unseen” data for the model, providing a final evaluation of the model’s performance. The test set is meant to simulate how the model would perform in the real world, and it’s crucial that it’s not used in any part of the training process to avoid data leakage. *For example, to evaluate the performance of a machine learning algorithm.*

Both validation and test datasets are essential for creating robust and reliable machine learning models, but they serve slightly different purposes in the process.

5.2 Validation Set Approach

Example: Auto data

A non-linear relationship between mpg and horsepower



5.2 Validation Set Approach

Example: Auto data

```
##### Comparing polynomial regression models using adjusted-R^2, AIC, BIC, Mallow's Cp

import numpy as np
import pandas as pd
import statsmodels.api as sm
from itertools import combinations
pd.set_option('display.max_columns',100)

# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset

# Validation approach
n = Auto.shape[0]
np.random.seed(1997)
Auto['horsepower2']=Auto['horsepower']**2
Auto['horsepower3']=Auto['horsepower']**3
Auto['horsepower4']=Auto['horsepower']**4
Auto['horsepower5']=Auto['horsepower']**5

# Convert to numpy arrays
df = Auto[['horsepower','horsepower2','horsepower3','horsepower4','horsepower5']]
y = Auto['mpg'].values

# Function to calculate Mallow's Cp
def mallows_cp(model, sigma_squared):
    p = model.df_model + 1
    rss = model.ssr
    return (rss / sigma_squared) - (len(y) - 2 * p)

# Fit models of different sizes
results = []

for k in range(1, 5):
    combo=df.columns[:k].tolist()
    X_subset = sm.add_constant(df[combo])
    model = sm.OLS(y, X_subset).fit()

    # Calculate statistics
    aic = model.aic
    bic = model.bic
    adj_r_squared = model.rsquared_adj
    sigma_squared = model.mse_resid
    cp = mallows_cp(model, sigma_squared)

    results.append({
        'Predictors': combo,
        'AIC': aic,
        'BIC': bic,
        'Mallow's Cp': cp,
        'Adjusted R^2': adj_r_squared
    })

# Convert results to DataFrame
results_df = pd.DataFrame(results)
print(results_df.sort_values('AIC').reset_index(drop=True))
```

	Predictors	AIC \
0	[horsepower, horsepower2]	2272.353522
1	[horsepower, horsepower2, horsepower3]	2273.531297
2	[horsepower, horsepower2, horsepower3, horsepo...	2274.108110
3	[horsepower]	2361.323658

	BIC	Mallow's Cp	Adjusted R^2
0	2284.267308	3.0	0.685953
1	2289.416344	4.0	0.685803
2	2293.964419	5.0	0.686133
3	2369.266182	2.0	0.604938

5.2 Validation Set Approach

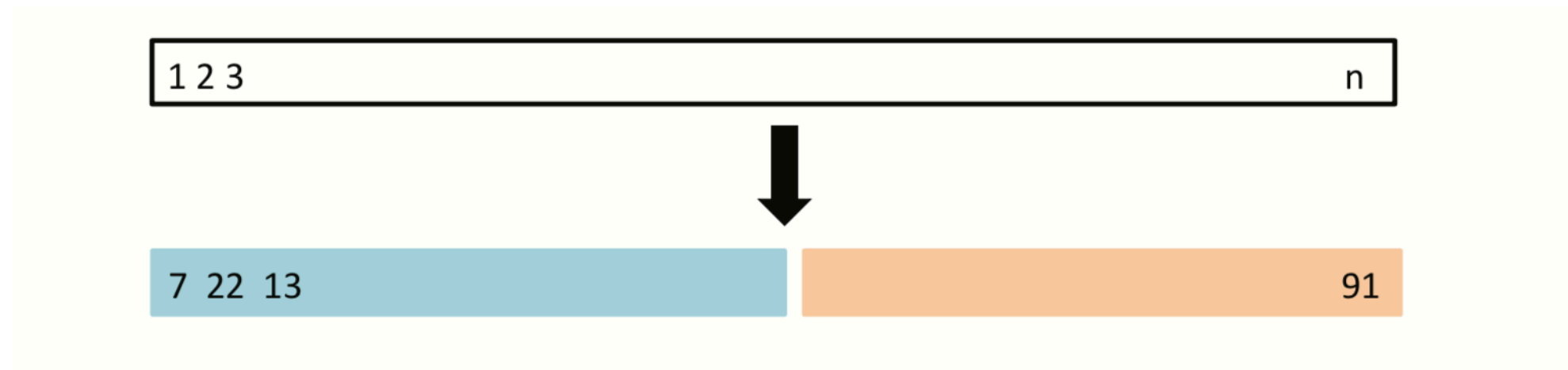


Figure: A schematic display of the validation set approach. A set of n observations are randomly split into a training set (shown in blue, containing observations 7, 22, and 13, among others) and a validation set (shown in beige, and containing observation 91, among others). The statistical learning method is fit on the training set, and its performance is evaluated on the validation set.

5.2 Validation Set Approach

Example: Auto data

- ▶ *A non-linear relationship between mpg and horsepower*
- ▶ $\text{mpg} \sim \text{horsepower} + \text{horsepower}^2$ is better than $\text{mpg} \sim \text{horsepower}$.
- ▶ *Should we add higher terms into the model?, like as cubic or even higher?*
- ▶ *One can check the p-values of regression coefficients to answer the question.*
- ▶ *In fact, a model selection problem, and we can use validation set approach.*

5.2 Validation Set Approach

Example: Auto data

```
> ##### AUTO DATA
> library(ISLR)
> Auto[1:10,]
      mpg cylinders displacement horsepower weight acceleration year origin      name
1    18         8         307         130   3504         12.0    70      1 chevrolet chevelle malibu
2    15         8         350         165   3693         11.5    70      1      buick skylark 320
3    18         8         318         150   3436         11.0    70      1    plymouth satellite
4    16         8         304         150   3433         12.0    70      1          amc rebel sst
5    17         8         302         140   3449         10.5    70      1          ford torino
6    15         8         429         198   4341         10.0    70      1    ford galaxie 500
7    14         8         454         220   4354          9.0    70      1    chevrolet impala
8    14         8         440         215   4312          8.5    70      1    plymouth fury iii
9    14         8         455         225   4425         10.0    70      1    pontiac catalina
10   15         8         390         190   3850          8.5    70      1    amc ambassador dpl
```

```
> summary(Auto)
      mpg      cylinders      displacement      horsepower      weight      acceleration      year      origin      name
Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.   :1613   Min.   : 8.00   Min.   :70.00   Min.   :1.000   amc matador      : 5
1st Qu.:17.00   1st Qu.:4.000   1st Qu.:105.0   1st Qu.: 75.0   1st Qu.:2225   1st Qu.:13.78   1st Qu.:73.00   1st Qu.:1.000   ford pinto       : 5
Median :22.75   Median :4.000   Median :151.0   Median : 93.5   Median :2804   Median :15.50   Median :76.00   Median :1.000   toyota corolla   : 5
Mean   :23.45   Mean   :5.472   Mean   :194.4   Mean   :104.5   Mean   :2978   Mean   :15.54   Mean   :75.98   Mean   :1.577   amc gremlin      : 4
3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:126.0   3rd Qu.:3615   3rd Qu.:17.02   3rd Qu.:79.00   3rd Qu.:2.000   amc hornet       : 4
Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.   :5140   Max.   :24.80   Max.   :82.00   Max.   :3.000   chevrolet chevette: 4
                                           (Other)          :365
```

5.2 Validation Set Approach

Example: Auto data

- ▶ *randomly split the 392 observations into two sets:*
- ▶ *a **training set** containing 196 of the data points, and a **validation set** containing the remaining 196 observations.*
- ▶ *fit various regression models on the training sample*
- ▶ *The validation set error rates result from evaluating their performance on the validation sample.*
- ▶ *Here we use MSE as a measure of validation set error, are shown in the Next slide*

5.2 Validation Set Approach

Example: Auto data in R

Linear Model

```
> set.seed(1)                                ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)                     ## split for train and test
> lm.fit=lm(mpg~horsepower ,data=Auto,subset=train)  ## train a linear model using the subset of Auto data
>
> attach(Auto)
The following object is masked from package:ggplot2:

    mpg

> mean((mpg-predict(lm.fit,Auto))[-train]^2)    ## average prediction error on test data
[1] 23.26601
```

```
> set.seed(2022)                             ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)                     ## split for train and test
> lm.fit=lm(mpg~horsepower ,data=Auto,subset=train)  ## train a linear model using the subset of Auto data
> mean((mpg-predict(lm.fit,Auto))[-train]^2)    ## average prediction error on test data
[1] 25.92479
```

```
> set.seed(1997)                             ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)                     ## split for train and test
> lm.fit=lm(mpg~horsepower ,data=Auto,subset=train)  ## train a linear model using the subset of Auto data
> mean((mpg-predict(lm.fit,Auto))[-train]^2)    ## average prediction error on test data
[1] 24.35463
```

5.2 Validation Set Approach

Example: Auto data in R

Quadratic Model

```
> set.seed(1)                                     ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit2=lm(mpg~poly(horsepower ,2),data=Auto,subset=train)    ## quadratic regression
> mean((mpg-predict(lm.fit2,Auto))[-train]^2)                  ## now we fit a polynomial regression and get the average prediction error
[1] 18.71646
```

```
> set.seed(2022)                                   ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit2=lm(mpg~poly(horsepower ,2),data=Auto,subset=train)    ## quadratic regression
> mean((mpg-predict(lm.fit2,Auto))[-train]^2)                  ## now we fit a polynomial regression and get the average prediction error
[1] 19.70661
```

```
> set.seed(1997)                                   ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit2=lm(mpg~poly(horsepower ,2),data=Auto,subset=train)    ## quadratic regression
> mean((mpg-predict(lm.fit2,Auto))[-train]^2)                  ## now we fit a polynomial regression and get the average prediction error
[1] 19.60572
```

5.2 Validation Set Approach

Example: Auto data in R

Cubic Model

```
> set.seed(1)                                     ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit3=lm(mpg~poly(horsepower ,3),data=Auto,subset=train)    ## cubic regression
> mean((mpg-predict(lm.fit3,Auto))[-train]^2)                  ## average prediction error
[1] 18.79401
```

```
> set.seed(2022)                                   ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit3=lm(mpg~poly(horsepower ,3),data=Auto,subset=train)    ## cubic regression
> mean((mpg-predict(lm.fit3,Auto))[-train]^2)                  ## average prediction error
[1] 19.81508
```

```
> set.seed(1997)                                   ## set the randomness seed for reproducibility in the future
> train=sample(392, 196)
> lm.fit3=lm(mpg~poly(horsepower ,3),data=Auto,subset=train)    ## cubic regression
> mean((mpg-predict(lm.fit3,Auto))[-train]^2)                  ## average prediction error
[1] 19.77751
```

5.2 Validation Set Approach

Example: Auto data in Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.datasets import fetch_openml

# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset
# Display the first 10 rows
print(Auto.head(10))

# Validation approach
n = Auto.shape[0]
np.random.seed(1997)
trainid = np.random.choice(n, n // 2, replace=False)

# Convert to numpy arrays
X = Auto[['horsepower']].values
y = Auto['mpg'].values

# Linear regression
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.5, random_state=1997)
```



```
# Sort the data
sorted_indices = np.argsort(X, axis=0).flatten()
X_sorted = X[sorted_indices]
y_sorted = y[sorted_indices]

# Function to fit and plot polynomial regression
def plot_polynomial_regression(X, y, degree):
    poly = PolynomialFeatures(degree=degree)
    X_poly = poly.fit_transform(X)
    lin_reg = LinearRegression()
    lin_reg.fit(X_poly, y)
    y_pred = lin_reg.predict(X_poly)
    return y_pred

# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(X_sorted, y_sorted, color='blue', label='Original data', alpha=0.5)

# Linear regression
y_pred_linear = plot_polynomial_regression(X_sorted, y_sorted, degree=1)
plt.plot(X_sorted, y_pred_linear, color='red', label='Linear fit', linewidth=2)

# Quadratic regression
y_pred_quadratic = plot_polynomial_regression(X_sorted, y_sorted, degree=2)
plt.plot(X_sorted, y_pred_quadratic, color='green', label='Quadratic fit', linewidth=2)

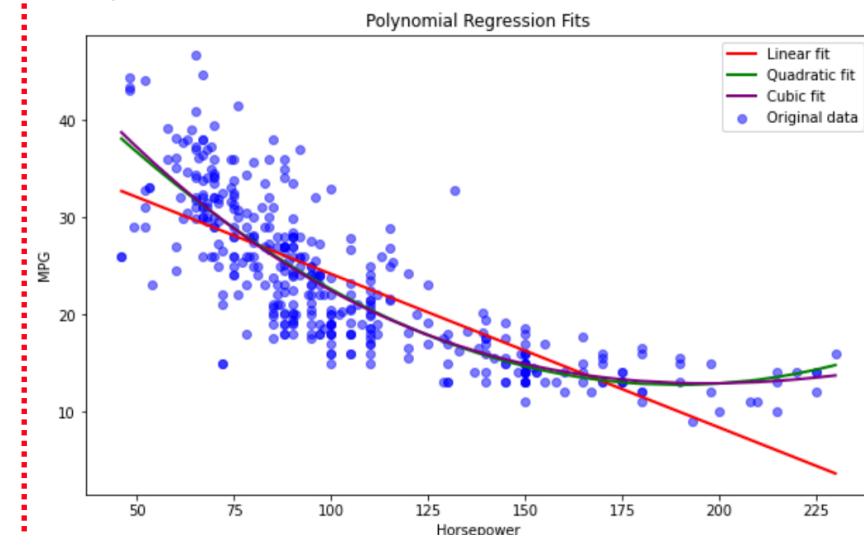
# Cubic regression
y_pred_cubic = plot_polynomial_regression(X_sorted, y_sorted, degree=3)
plt.plot(X_sorted, y_pred_cubic, color='purple', label='Cubic fit', linewidth=2)

# Labels and legend
plt.xlabel('Horsepower')
plt.ylabel('MPG')
plt.title('Polynomial Regression Fits')
plt.legend()
plt.show()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	\
0	18.0	8	307.0	130	3504	12.0	70	
1	15.0	8	350.0	165	3693	11.5	70	
2	18.0	8	318.0	150	3436	11.0	70	
3	16.0	8	304.0	150	3433	12.0	70	
4	17.0	8	302.0	140	3449	10.5	70	
5	15.0	8	429.0	198	4341	10.0	70	
6	14.0	8	454.0	220	4354	9.0	70	
7	14.0	8	440.0	215	4312	8.5	70	
8	14.0	8	455.0	225	4425	10.0	70	
9	15.0	8	390.0	190	3850	8.5	70	

origin	name
0	1 chevrolet chevelle malibu
1	1 buick skylark 320
2	1 plymouth satellite
3	1 amc rebel sst
4	1 ford torino
5	1 ford galaxie 500
6	1 chevrolet impala
7	1 plymouth fury iii
8	1 pontiac catalina
9	1 amc ambassador dpl

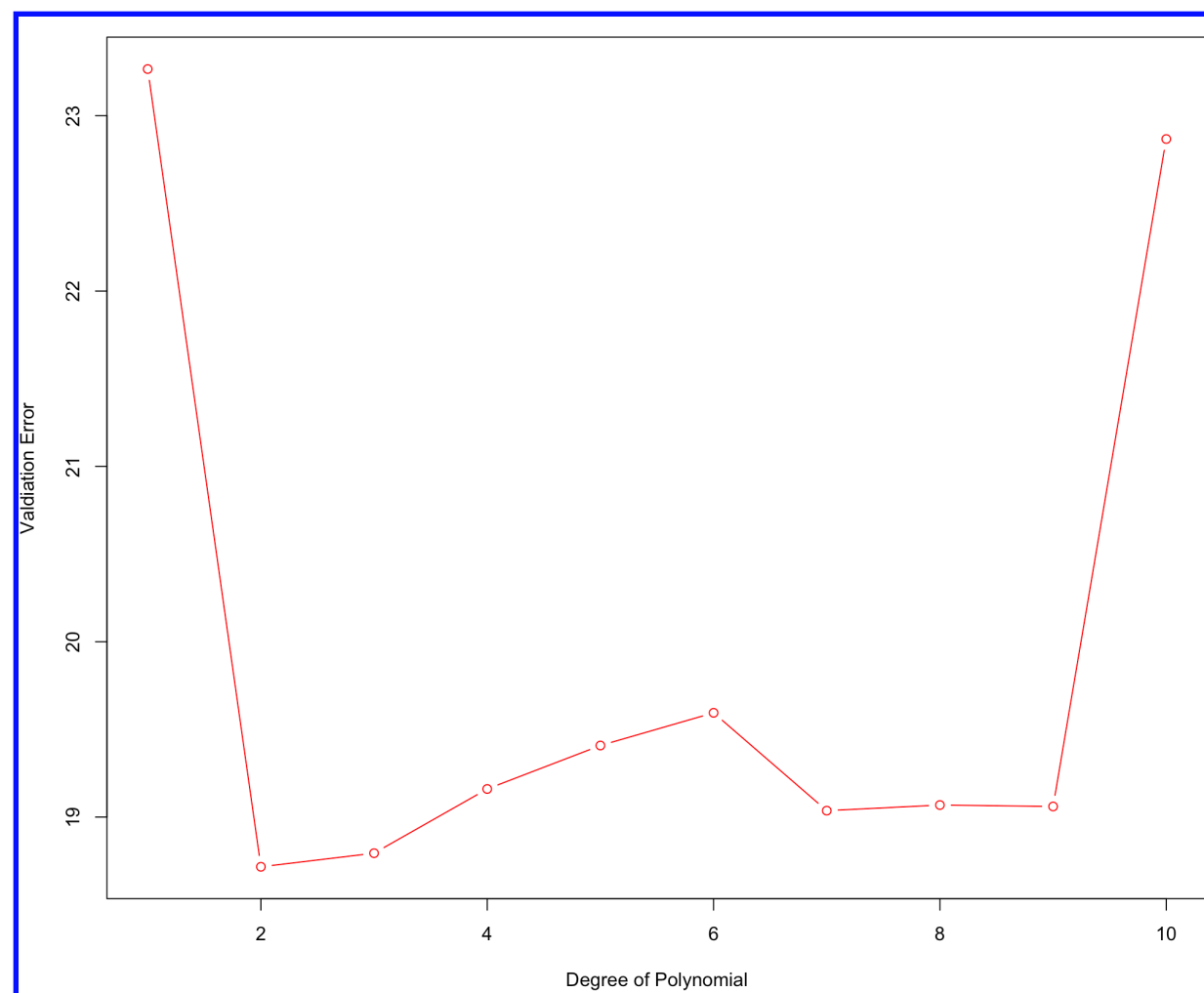
Mean squared error (linear): 26.42140809416316
Mean squared error (quadratic): 21.56632676442944
Mean squared error (cubic): 21.51563030957688



5.2 Validation Set Approach

Example: Auto data in R

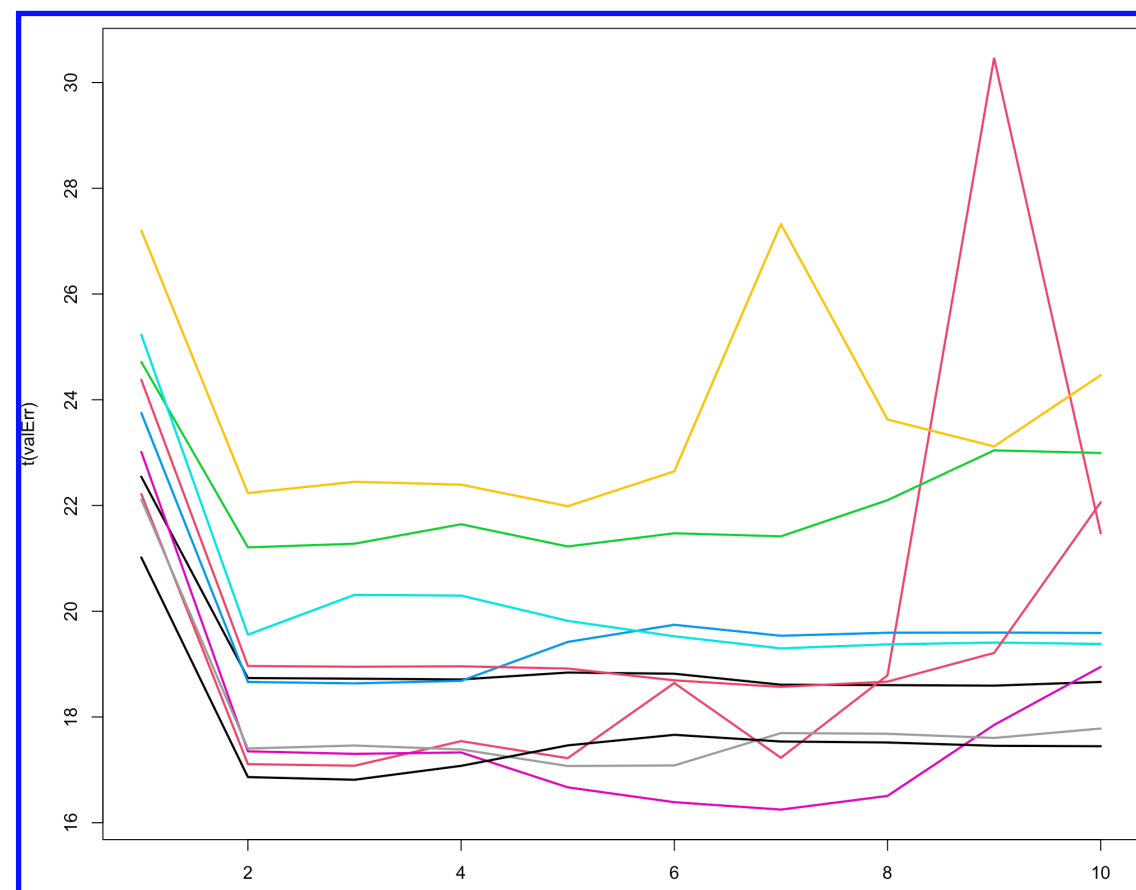
```
##### draw the plot
valErr<-rep(0,10)
set.seed(1)                                     ## set the randomness seed for reproducibility in the future
train=sample(392, 196)                         ## split for train and test
for(k in 1:10){
  lm.fit=lm(mpg~poly(horsepower ,k),data=Auto,subset=train)    ## cubic regression
  valErr[k]<-mean((mpg-predict(lm.fit,Auto))[-train]^2)         ## average prediction error
}
plot(1:10,valErr,type='b',col='red',xlab='Degree of Polynomial',ylab='Valdiation Error')
```



5.2 Validation Set Approach

Example: Auto data in R

```
##### draw the plot
valErr<-matrix(rep(0,100),10,10)
for(j in 1:10){
  set.seed(j+2000)                                     ## set the randomness seed for reproducibility in the future
  train=sample(392, 196)                                ## split for train and test
  for(k in 1:10){
    lm.fit=lm(mpg~poly(horsepower ,k),data=Auto,subset=train)    ## cubic regression
    valErr[j,k]<-mean((mpg-predict(lm.fit,Auto))[-train]^2)        ## average prediction error
  }
}
matplot(t(valErr), type = "l",
        col = 1:10, lty = 1, lwd = 2)
```



5.2 Validation Set Approach

Example: Auto data

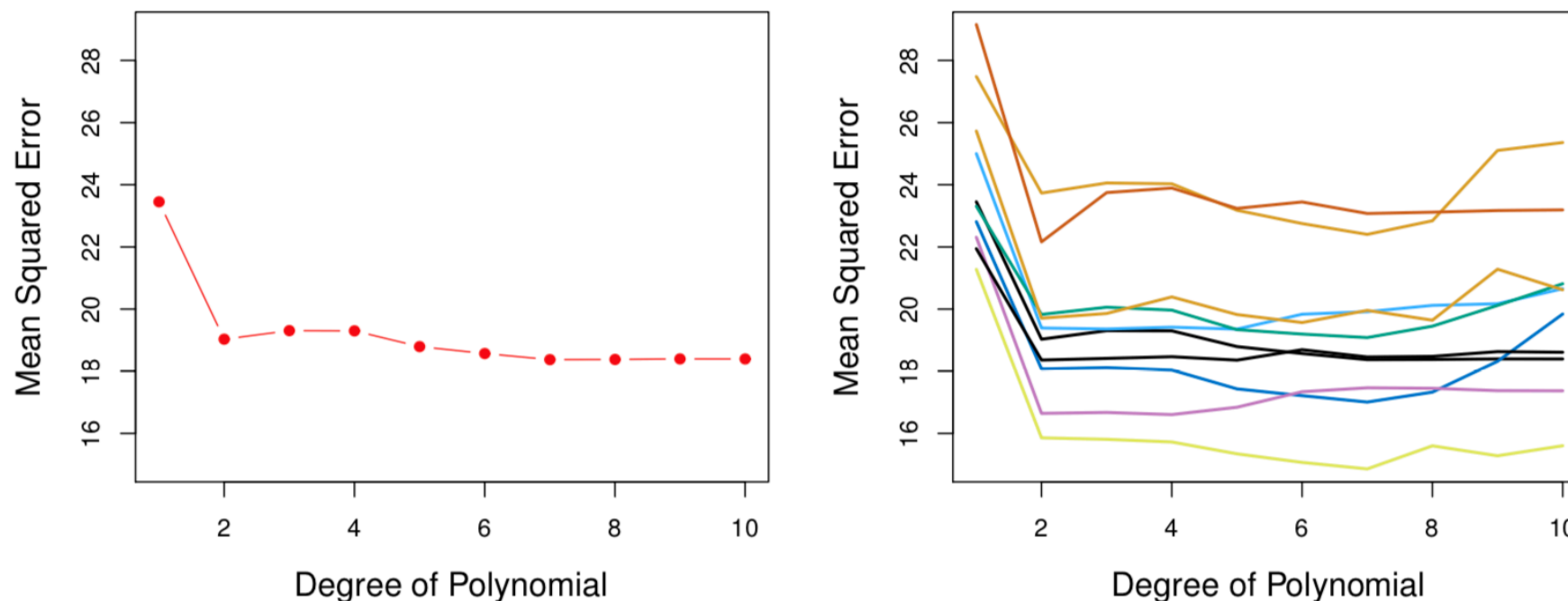


Figure: The validation set approach was used on the Auto data set in order to estimate the test error that results from predicting mpg using polynomial functions of horsepower. Left: Validation error estimates for a single split into training and validation data sets. Right: The validation method was repeated ten times, each time using a different random split of the observations into a training set and a validation set. This illustrates the variability in the estimated test MSE that results from this approach.

5.2 Validation Set Approach

Example: Auto data

▶ *The validation set MSE for the quadratic fit is considerably smaller than for the linear fit.*

▶ *Validation set MSE for the cubic fit is actually slightly larger than for the quadratic fit.*

▶ *This implies that including a cubic term in the regression does NOT lead to better prediction than simply using a quadratic term.*

▶ *Repeat the process of randomly splitting the sample set into two parts, we will get a somewhat different estimate for the test MSE.*

5.2 Validation Set Approach

Example: Auto data

- ▶ *A quadratic term has a dramatically smaller validation set MSE than the model with only a linear term.*
- ▶ *Not much benefit in including cubic or higher-order polynomial terms in the model.*
- ▶ *Each of the ten curves results in a different test MSE estimate for each of the ten regression models considered.*
- ▶ *No consensus among the curves as to which model results in the smallest validation set MSE.*
- ▶ *Based on the variability among these curves, all that we can conclude with any confidence is that the linear fit is not adequate for this data.*
- ▶ *The validation set approach is conceptually simple and is easy to implement.*

5.2 Validation Set Approach

Summary

- ▶ *The validation estimate of the test error rate can be highly variable, depending on the random split.*
- ▶ *Only a subset of the observations—the training set are used to fit the model.*
- ▶ *Statistical methods tend to perform worse when trained on fewer observations.*
- ▶ *The validation set error rate may tend to overestimate the test error rate for the model fit on the entire data set.*

5.3 Cross Validation

Overcome the drawback of validation set approach

- ▶ *Our ultimate goal is to produce the best model with best prediction accuracy.*
- ▶ *Validation set approach has a drawback of using ONLY training data to fit model.*
- ▶ *The validation data do not participate in model building but only model assessment.*
- ▶ *A “waste” of data. We need more data to participate in model building.*

5.3 Cross Validation

Drawback of validation set approach

- ▶ *It may over-estimate the test error for the model with all data used to fit.*
- ▶ *Statistical methods tend to perform worse when trained on fewer observations.*
- ▶ *The validation set error rate may tend to overestimate the test error rate for the model fit on the entire data set.*
- ▶ *The method of cross validation could overcome these drawbacks, by effectively using EVERY data point in model building!*

5.3 Cross Validation

Leave-one-out cross-validation

► Suppose the data contain n data points.

► First, pick data point 1 as validation set, the rest as training set. fit the model on the training set, evaluate the test error, on the validation set, denoted as say MSE_1 .

► Second, pick data point 2 as validation set, the rest as training set. fit the model on the training set, evaluate the test error on the validation set, denoted as say MSE_2 .

► (repeat the procedure for all data point.)

► Obtain an estimate of the test error by combining the MSE_i , $i = 1, \dots, n$:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n MSE_i$$

5.3 Cross Validation

LOOCV

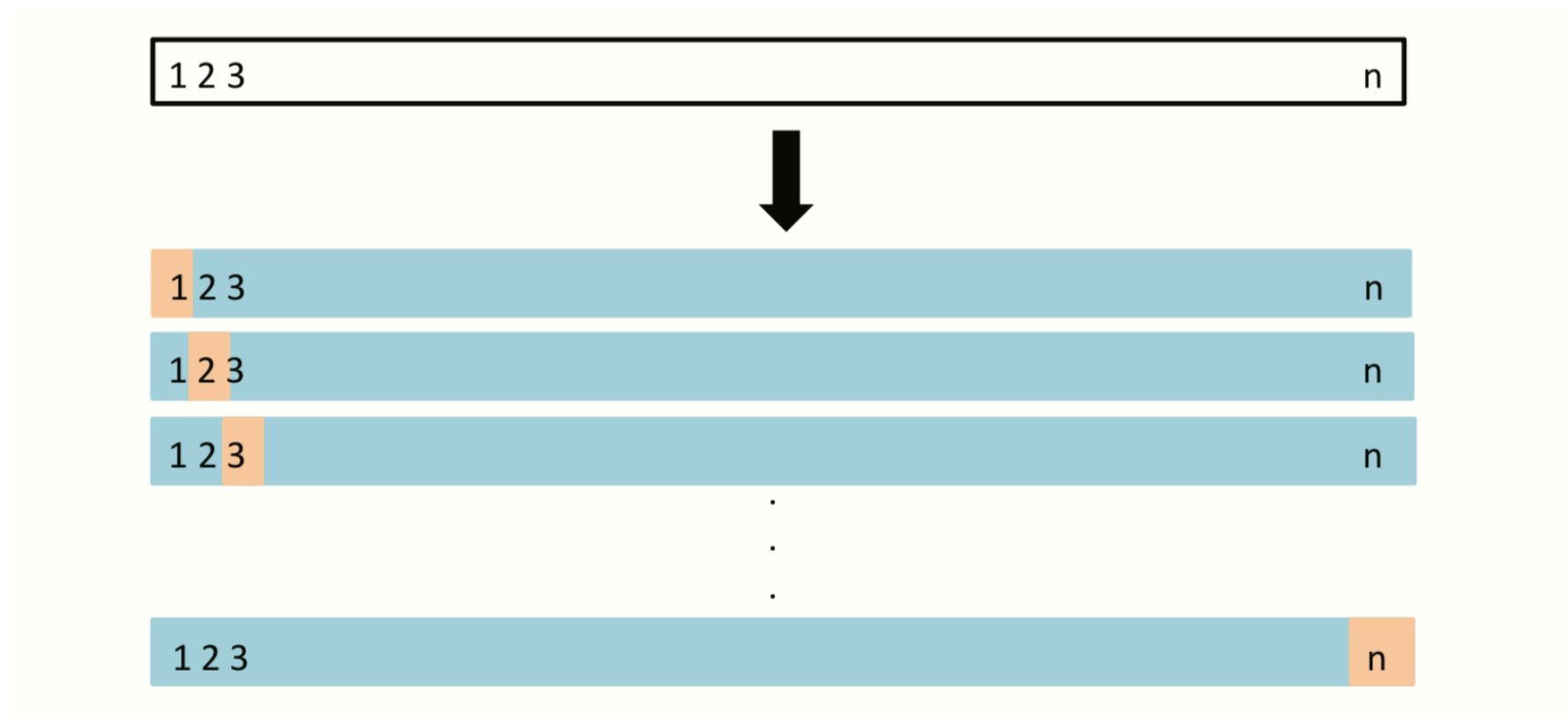


Figure: 5.3. A schematic display of LOOCV. A set of n data points is repeatedly split into a training set (shown in blue) containing all but one observation, and a validation set that contains only that observation (shown in beige). The test error is then estimated by averaging the n resulting MSE's. The first training set contains all but observation 1, the second training set contains all but observation 2, and so forth.

5.3 Cross Validation

Advantages of LOOCV

- ▶ *Far less bias, since the training data size ($n - 1$) is close to the entire data size (n).*
- ▶ *One single test error estimate (thanks to the averaging), without the variability validation set approach.*
- ▶ *A disadvantage: could be computationally expensive since the model need to be fit n times.*
- ▶ The MSE_i may be too much correlated.

5.3 Cross Validation

L00CV on Auto data in Python

```
##### L00C on auto data

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.pipeline import make_pipeline

# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset

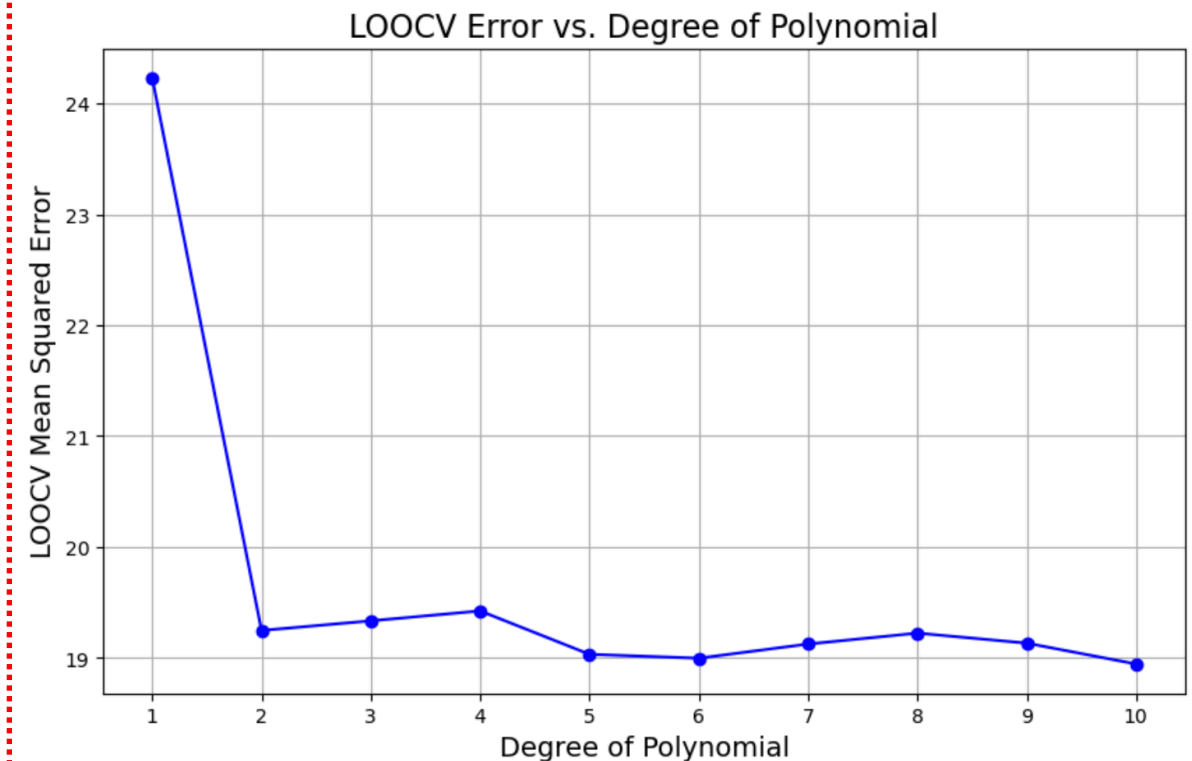
##### Linear model and L00C
# Define features and target variable
X = Auto[['horsepower']].values # Predictor
y = Auto['mpg'].values # Response
linear_model = LinearRegression()
linear_model.fit(X, y)
print("Linear Regression Coefficients:")
print(f"Intercept: {linear_model.intercept_}")
print(f"Slope (horsepower): {linear_model.coef_[0]}")

loo = LeaveOneOut()
cv_errors = cross_val_score(linear_model, X, y, cv=loo, scoring='neg_mean_squared_error')
mse = -cv_errors
loocv_error = mse.mean()
print(f"L00CV Estimate of Prediction Error (MSE): {loocv_error:.4f}")

##### Polynomial regression and L00C
degrees = range(1, 11)
cv_errors_poly = []
for degree in degrees:
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    scores = cross_val_score(model, X, y, cv=loo, scoring='neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_errors_poly.append(avg_mse)
    print(f"Degree {degree}: L00CV MSE = {avg_mse:.4f}")

##### Visualization
plt.figure(figsize=(10, 6))
plt.plot(degrees, cv_errors_poly, marker='o', linestyle='-', color='b')
plt.title('L00CV Error vs. Degree of Polynomial', fontsize=16)
plt.xlabel('Degree of Polynomial', fontsize=14)
plt.ylabel('L00CV Mean Squared Error', fontsize=14)
plt.xticks(degrees)
plt.grid(True)
plt.show()
```

Linear Regression Coefficients:
Intercept: 39.93586102117047
Slope (horsepower): -0.15784473335365365
L00CV Estimate of Prediction Error (MSE): 24.2315
Degree 1: L00CV MSE = 24.2315
Degree 2: L00CV MSE = 19.2482
Degree 3: L00CV MSE = 19.3350
Degree 4: L00CV MSE = 19.4244
Degree 5: L00CV MSE = 19.0332
Degree 6: L00CV MSE = 18.9980
Degree 7: L00CV MSE = 19.1259
Degree 8: L00CV MSE = 19.2242
Degree 9: L00CV MSE = 19.1340
Degree 10: L00CV MSE = 18.9457



5.3 Cross Validation

L00CV on Auto data in R

```
> #####
> ##### Leave-One-Out Cross Validation
> #####
> ## The L00CV estimate can be automatically computed for any generalized linear model using the glm() and cv.glm() functions.
> glm.fit=glm(mpg~horsepower ,data=Auto)      ## fit a linear regression model with "family=...", identical to lm(mpg~horsepower, data=Auto)
> coef(glm.fit)                                ## check the coefficients
(Intercept)  horsepower
 39.9358610   -0.1578447
>
> library(boot)                                ##The cv.glm() function is part of the boot library.

Attaching package: 'boot'

The following object is masked from 'package:lattice':

    melanoma

> glm.fit=glm(mpg~horsepower ,data=Auto)
> cv.err=cv.glm(Auto,glm.fit)                  ## L00CV estimate of prediction error
> cv.err$delta                                ## The two numbers in the delta vector contain the cross-validation results.
[1] 24.23151 24.23114
```

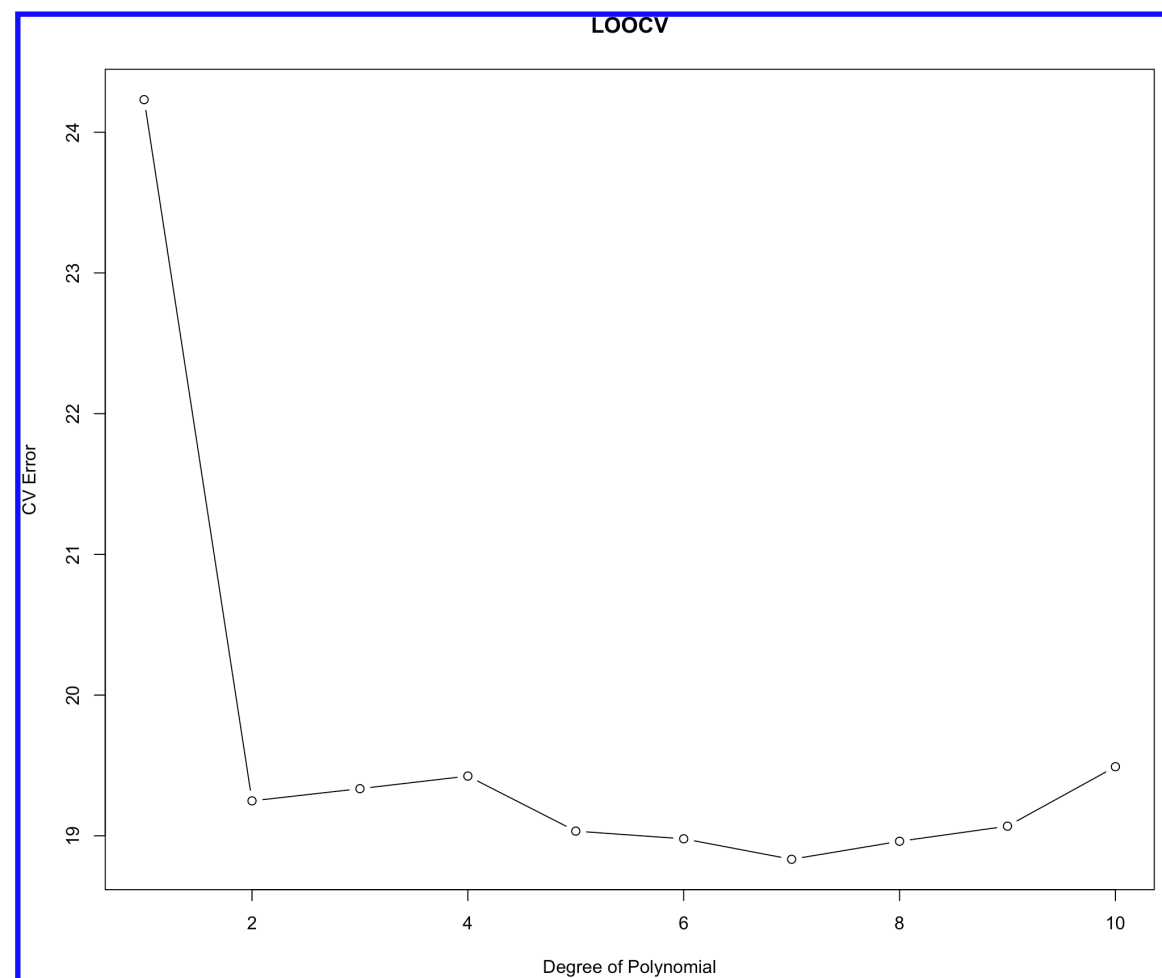
raw CV error

adjusted CV error to compensate the bias

5.3 Cross Validation

L00CV on Auto data in R

```
> ### we repeat the procedure for increasingly complex polynomial fits. We use for() to automate the process.  
> cv.error=rep(0,10)  
> for (i in 1:10){  
+   glm.fit=glm(mpg~poly(horsepower ,i),data=Auto)  
+   cv.error[i]=cv.glm(Auto,glm.fit)$delta[1]}  
> cv.error  
[1] 24.23151 19.24821 19.33498 19.42443 19.03321 18.97864 18.83305 18.96115 19.06863 19.49093  
> plot(1:10,cv.error,type='b',xlab="Degree of Polynomial",ylab="CV Error",main="L00CV")
```



5.3 Cross Validation

LOOCV on Auto data

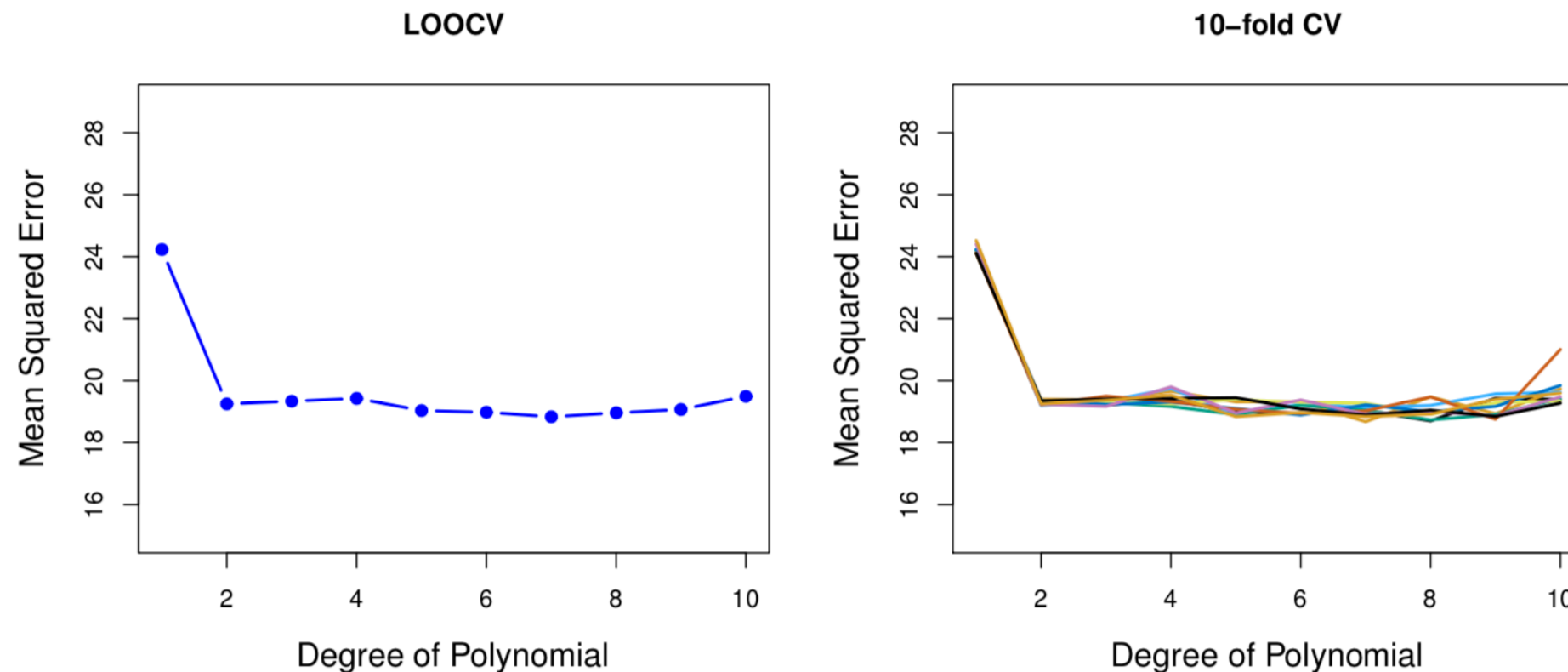


Figure: 5.4. Cross-validation was used on the Auto data set in order to estimate the test error that results from predicting mpg using polynomial functions of horsepower. Left: The LOOCV error curve. Right: 10-fold CV was run nine separate times, each with a different random split of the data into ten parts. The figure shows the nine slightly different CV error curves.

5.3 Cross Validation

Complexity of LOOCV in linear model

Consider linear model:

$$y_i = \mathbf{x}_i^T \beta + \epsilon_i, \quad i = 1, \dots, n$$

and the fitted values $\hat{y}_i = \mathbf{x}_i^T \hat{\beta}$, where $\hat{\beta}$ is the least squares estimate of β based on all data $(\mathbf{x}_i, y_i), i = 1, \dots, n$.

Using LOOCV, the

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{(i)})^2$$

where $\hat{y}_i^{(i)} = \mathbf{x}_i^T \hat{\beta}^{(i)}$ is the model predictor of y_i based on the linear model fitted by all data except $(\mathbf{x}_i, y_i)$ (delete one), i.e., $\hat{\beta}^{(i)}$ is the least squares estimate of β based on all data but $(\mathbf{x}_i, y_i)$.

5.3 Cross Validation

Simpler way of LOOCV in linear model

▶ *The formula on last slide looks to be complicated to compute least squares estimate n times.*

▶ Easy formula:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2$$

where $\hat{y}_i$ is the fitted values of least squares method based on all data. h_i is the leverage.

What is h_i ?

→ next slide

5.3 Cross Validation

Recall: Leverage

Recall the hat matrix

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \quad \text{as } \hat{y} = \mathbf{H}y.$$

Let $h_{ij} = \mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_j$ be the (i, j) elements of $\mathbf{H}$.

The leverage of the i -th observation is just the i -th diagonal element of $\mathbf{H}$, denoted as h_{ii} .

A high leverage implies that observation is quite influential. Note that the average of h_{ii} is $(p + 1)/n$.

E.g., if h_{ii} is greater than $2(p + 1)/n$, twice of the average, is generally considered large.

5.3 Cross Validation

Simplicity of LOOCV in linear model

- ▶ *One fit (with all data) does it all!*
- ▶ *The prediction error rate (in terms of MSE) is just weighted average of the least squares fit residuals.*
- ▶ *High leverage point gets more weight in prediction error estimation.*

5.3 Cross Validation

Fast computation of LOOCV

The leave-one-out cross-validation statistic is given by

$$CV = \frac{1}{N} \sum_{i=1}^N e_{[i]}^2,$$

where $e_{[i]} = y_i - \hat{y}_{[i]}$, the observations are given by $y_1, \dots, y_N$, and $\hat{y}_{[i]}$ is the predicted value obtained when the model is estimated with the i th case deleted.

Suppose we have a linear regression model $\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{e}$. The $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$ and $\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ is the hat matrix. It has this name because it is used to compute $\hat{\mathbf{Y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{Y}$. If the diagonal values of $\mathbf{H}$ are denoted by $h_1, \dots, h_N$, then the leave-one-out cross-validation statistic can be computed using

$$CV = \frac{1}{N} \sum_{i=1}^N [e_i / (1 - h_i)]^2,$$

where $e_i = y_i - \hat{y}_i$ is predicted value obtained when the model is estimated with all data included

5.3 Cross Validation

Fast computation of LOOCV

Proof of the easier formula

Let $\mathbf{X}_{[i]}$ and $\mathbf{Y}_{[i]}$ be similar to $\mathbf{X}$ and $\mathbf{Y}$ but with the i th row deleted in each case. Let $\mathbf{x}_i^T$ be the i th row of $\mathbf{X}$ and let

$$1) \quad \hat{\boldsymbol{\beta}}_{[i]} = (\mathbf{X}_{[i]}^T \mathbf{X}_{[i]})^{-1} \mathbf{X}_{[i]}^T \mathbf{Y}_{[i]}$$

be the estimate of $\boldsymbol{\beta}$ without the i th case. Then $e_{[i]} = y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}_{[i]}$.

Now $\mathbf{X}_{[i]}^T \mathbf{X}_{[i]} = (\mathbf{X}^T \mathbf{X} - \mathbf{x}_i \mathbf{x}_i^T)$ and $\mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i = h_i$. So by the Sherman-Morrison-Woodbury formula,

2)

$$(\mathbf{X}_{[i]}^T \mathbf{X}_{[i]})^{-1} = (\mathbf{X}^T \mathbf{X})^{-1} + \frac{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i \mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1}}{1 - h_i}.$$

5.3 Cross Validation

Fast computation of LOOCV

Proof of the easier formula (cont.)

Also note that $\mathbf{X}_{[i]}^T \mathbf{Y}_{[i]} = \mathbf{X}^T \mathbf{Y} - \mathbf{x}_i y_i$. Therefore

$$\begin{aligned}
 3) \quad \hat{\beta}_{[i]} &= \left[(\mathbf{X}^T \mathbf{X})^{-1} + \frac{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i \mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1}}{1 - h_i} \right] (\mathbf{X}^T \mathbf{Y} - \mathbf{x}_i y_i) \\
 &= \hat{\beta} - \left[\frac{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i}{1 - h_i} \right] [y_i(1 - h_i) - \mathbf{x}_i^T \hat{\beta} + h_i y_i] \\
 &= \hat{\beta} - (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i e_i / (1 - h_i)
 \end{aligned}$$

$$\begin{aligned}
 4) \quad e_{[i]} &= y_i - \mathbf{x}_i^T \hat{\beta}_{[i]} \\
 &= y_i - \mathbf{x}_i^T \left[\hat{\beta} - (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i e_i / (1 - h_i) \right] \\
 &= e_i + h_i e_i / (1 - h_i) = e_i / (1 - h_i)
 \end{aligned}$$

5.3 Cross Validation

K-fold cross validation

▶ *Divide the data into K subsets, usually of equal or similar sizes (n/K).*

▶ Treat one subset as validation set, the rest together as a training set. Run the model fitting on training set. Calculate the test error estimate on the validation set, denoted as MSE_i , say.

▶ *Repeat the procedures over every subset.*

▶ Average over the above K estimates of the test errors, and obtain

$$CV_{(K)} = \frac{1}{K} \sum_{i=1}^K MSE_i$$

▶ *LOOCV is a special case of K -fold cross validation, actually n -fold cross validation.*

5.3 Cross Validation

K-fold cross validation

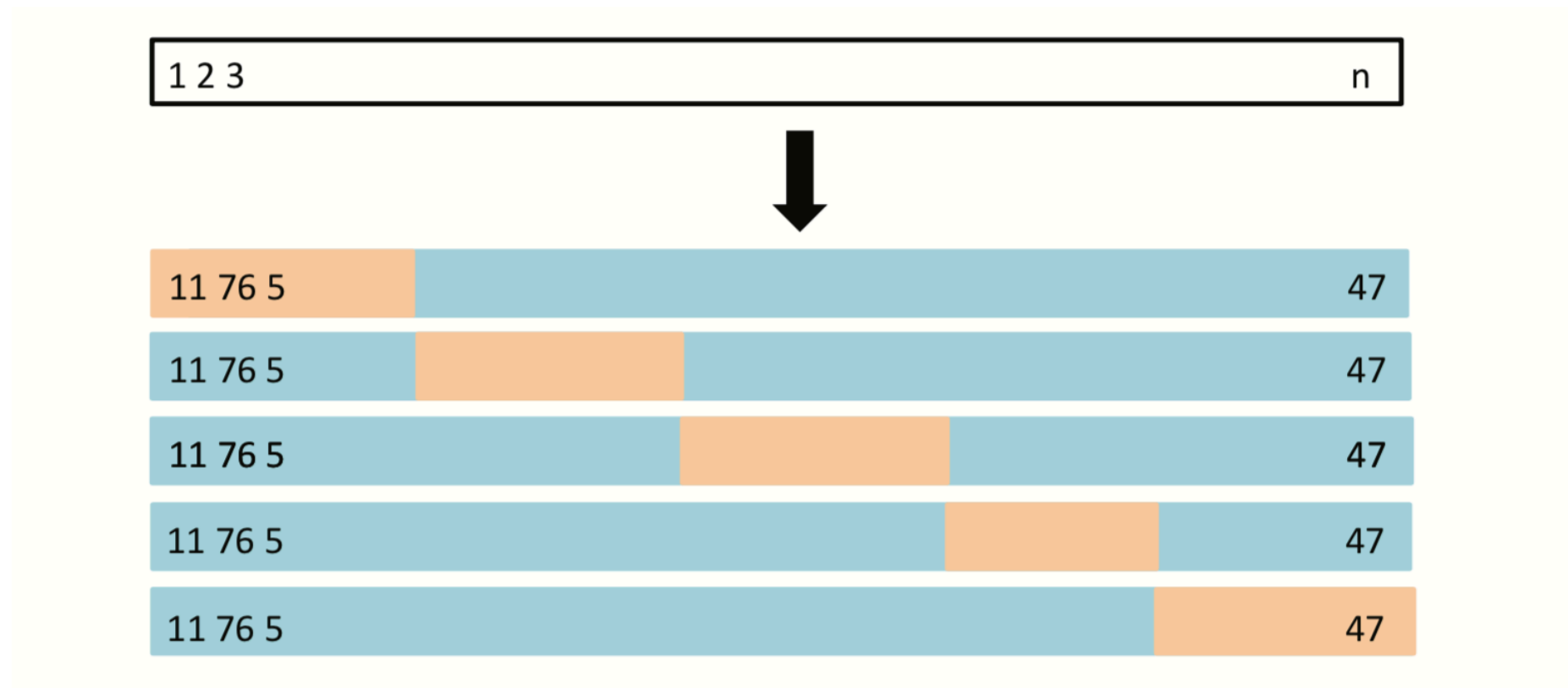


Figure: 5.5. A schematic display of 5-fold CV. A set of n observations is randomly split into five non-overlapping groups. Each of these fifths acts as a validation set (shown in beige), and the remainder as a training set (shown in blue). The test error is estimated by averaging the five resulting MSE estimates.

5.3 Cross Validation

K-fold cross validation

Common choices of K : $K=5$ or $K=10$.

Advantage over LOOCV:

1. computationally lighter, especially for complex model with large data.
2. Likely less variance (to be addressed later)

Advantage over validation set approach:

Less variability resulting from the data-split, thanks to the averaging.

5.3 Cross Validation

K-fold CV on Auto Data in Python

```
##### 10-fold CV on auto data
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import make_pipeline

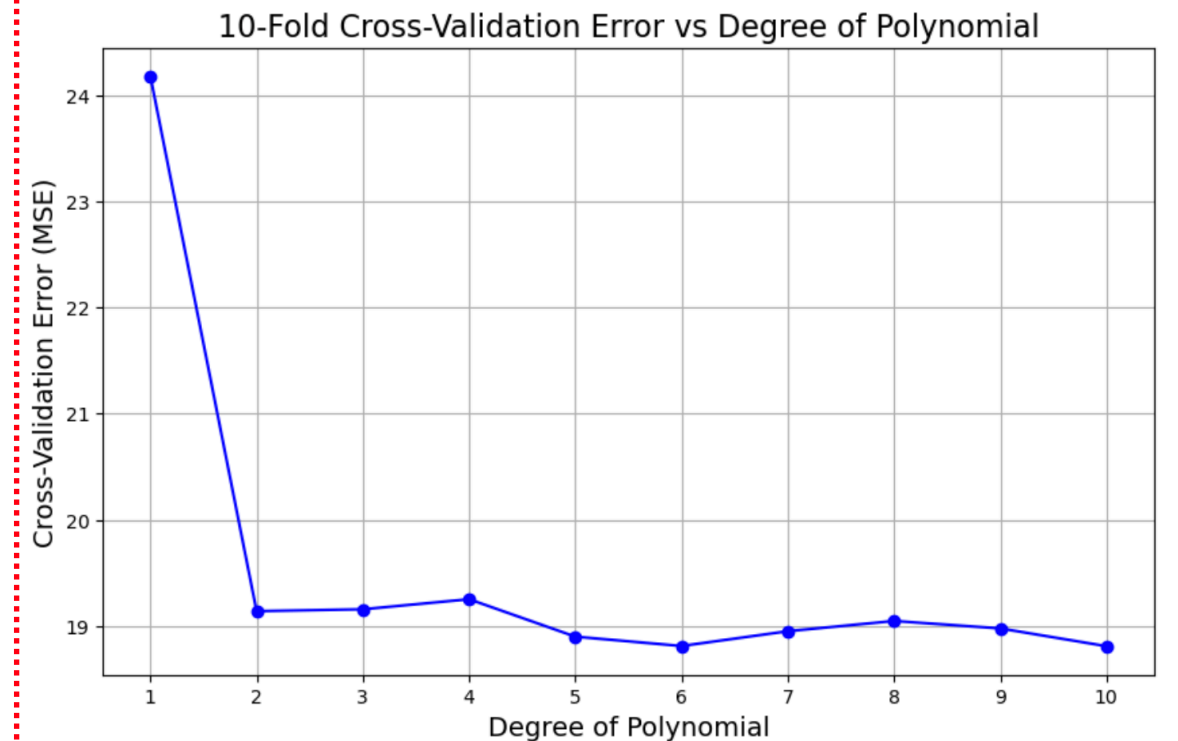
# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset

np.random.seed(17)
X = Auto[['horsepower']].values
y = Auto['mpg'].values
cv_error_10 = []

# Define 10-fold cross-validation
kfold = KFold(n_splits=10, shuffle=True, random_state=17)
for degree in range(1, 11):
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    scores = cross_val_score(model, X, y, cv=kfold, scoring='neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_error_10.append(avg_mse)
    print(f"Degree {degree}: 10-fold CV MSE = {avg_mse:.4f}")

##plot
degrees = range(1, 11)
plt.figure(figsize=(10, 6))
plt.plot(degrees, cv_error_10, marker='o', linestyle='-', color='blue')
plt.title('10-Fold Cross-Validation Error vs Degree of Polynomial', fontsize=16)
plt.xlabel('Degree of Polynomial', fontsize=14)
plt.ylabel('Cross-Validation Error (MSE)', fontsize=14)
plt.xticks(degrees)
plt.grid(True)
plt.show()
```

```
Degree 1: 10-fold CV MSE = 24.1823
Degree 2: 10-fold CV MSE = 19.1376
Degree 3: 10-fold CV MSE = 19.1552
Degree 4: 10-fold CV MSE = 19.2502
Degree 5: 10-fold CV MSE = 18.8971
Degree 6: 10-fold CV MSE = 18.8088
Degree 7: 10-fold CV MSE = 18.9483
Degree 8: 10-fold CV MSE = 19.0451
Degree 9: 10-fold CV MSE = 18.9747
Degree 10: 10-fold CV MSE = 18.8070
```



5.3 Cross Validation

K-fold CV on Auto Data in Python

```
##### 10-fold CV on auto data
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import make_pipeline

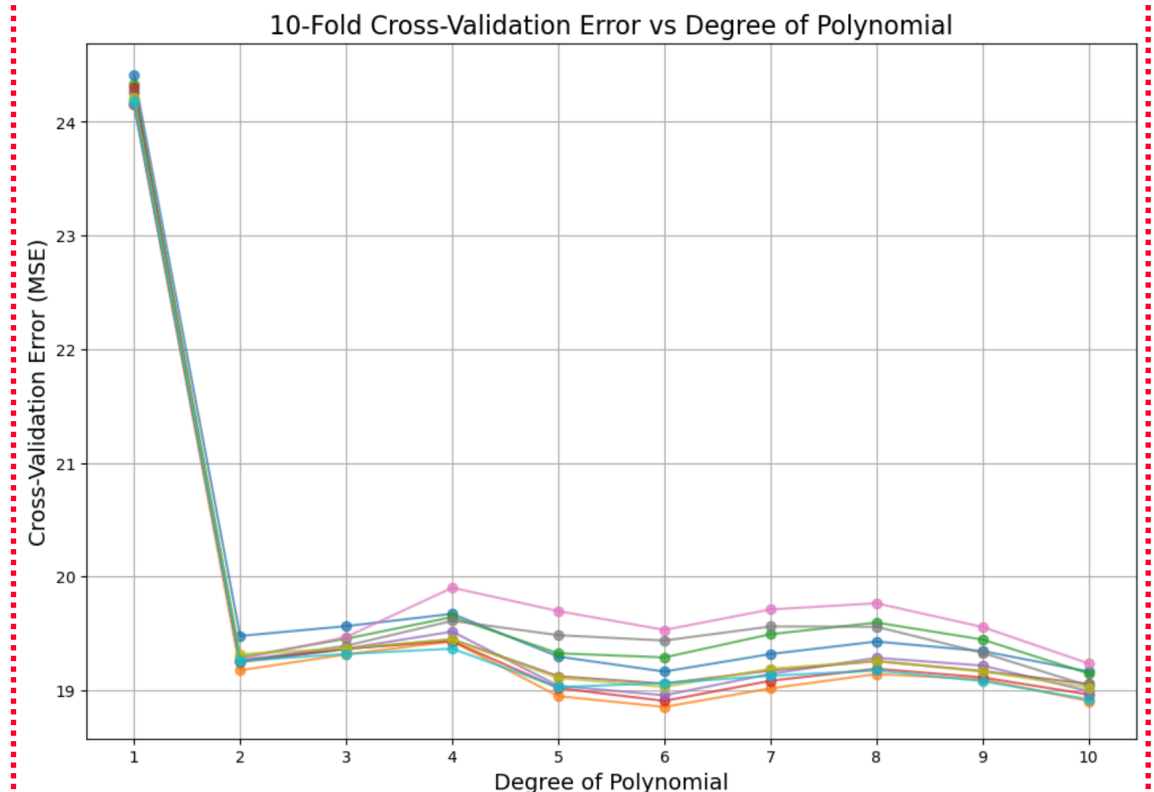
# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset

np.random.seed(17)
X = Auto[['horsepower']].values
y = Auto['mpg'].values

cv_error_10 = np.zeros((10, 10))
for j in range(10):
    seed = j + 2020
    np.random.seed(seed) # Set seed for reproducibility
    kfold = KFold(n_splits=10, shuffle=True, random_state=seed)
    for i in range(1, 11):
        model = make_pipeline(PolynomialFeatures(i), LinearRegression())
        scores = cross_val_score(model, X, y, cv=kfold, scoring='neg_mean_squared_error')
        avg_mse = -scores.mean()
        cv_error_10[i-1, j] = avg_mse
        #print(f"Seed {seed}, Degree {i}: 10-fold CV MSE = {avg_mse:.4f}")
print("\n10-fold CV Errors Matrix (Degrees 1-10, Seeds 2021-2030):")
print(cv_error_10)
### plot
degrees = range(1, 11)
plt.figure(figsize=(12, 8))
for j in range(10):
    plt.plot(degrees, cv_error_10[:, j], marker='o', linestyle='-', label=f'Seed {2021 + j}', alpha=0.7)
plt.title('10-Fold Cross-Validation Error vs Degree of Polynomial', fontsize=16)
plt.xlabel('Degree of Polynomial', fontsize=14)
plt.ylabel('Cross-Validation Error (MSE)', fontsize=14)
plt.xticks(degrees)
plt.grid(True)
plt.show()
```

10-fold CV Errors Matrix (Degrees 1-10, Seeds 2021-2030):

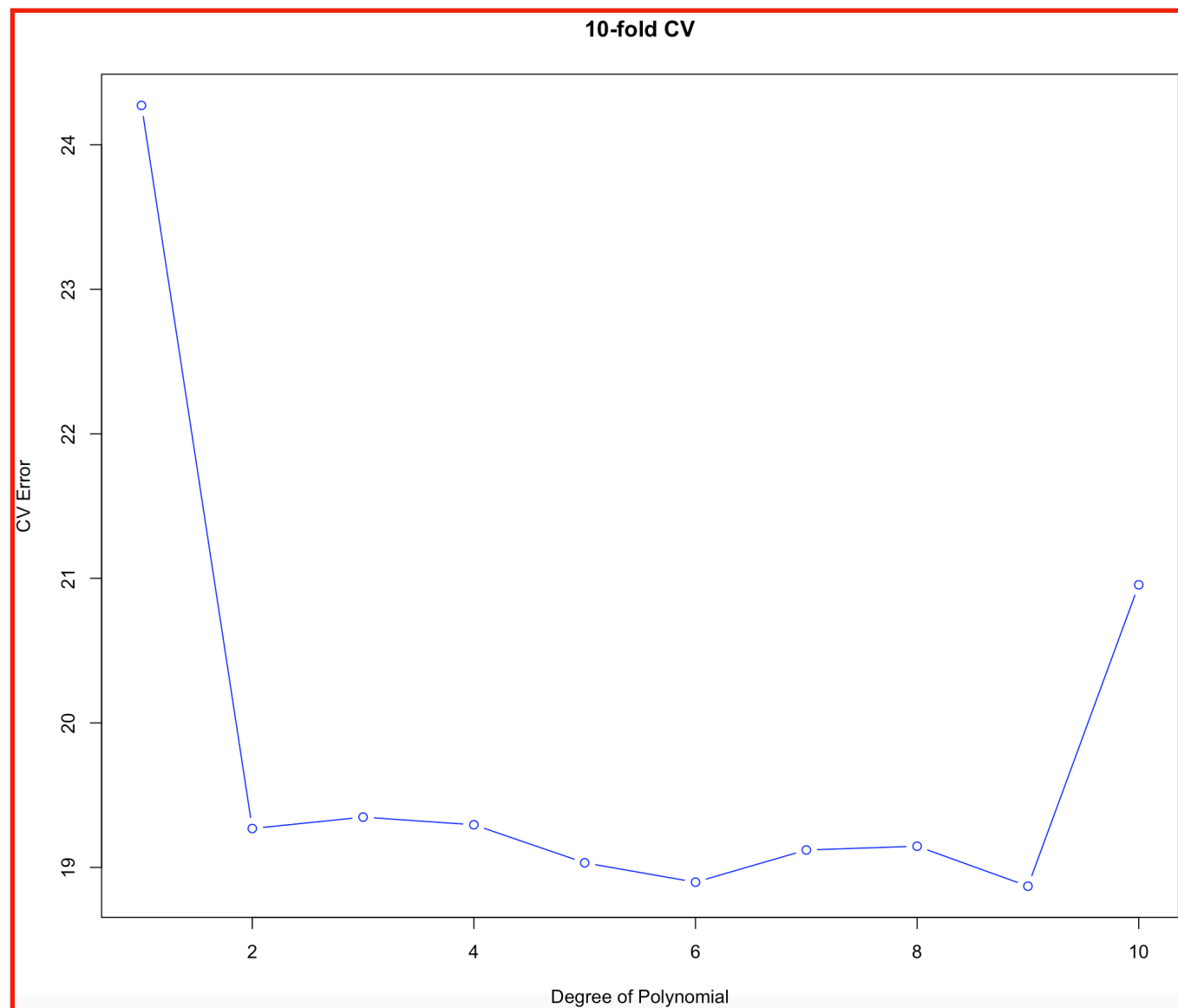
[24.41478628	24.15250509	24.3284091	24.30148874	24.15982413	24.26741769	24.15279153	24.1547865	24.21689802	24.18724963]
[19.47716253	19.17534998	19.29013934	19.26025709	19.25481252	19.24758108	19.27210662	19.25794479	19.31727319	19.25747397]
[19.5634159	19.31466214	19.45277302	19.3661635	19.3652956	19.36165981	19.46805008	19.39100016	19.3675152	19.31901631]
[19.67160951	19.43025088	19.64329392	19.4309647	19.51558761	19.44739552	19.90114923	19.61009481	19.44693041	19.365367]
[19.29526526	18.94739603	19.32440044	19.01823967	19.03811193	19.12260225	19.69605923	19.48400216	19.1075344	19.02611025]
[19.1635798	18.85277992	19.28841326	18.90560461	18.95509522	19.05731396	19.53018731	19.43693326	19.02883034	19.0634734]
[19.31819004	19.01580434	19.49247878	19.08275864	19.14275301	19.17331564	19.71027412	19.56044077	19.18898558	19.12666791]
[19.42793053	19.14199326	19.59478444	19.18830722	19.28451172	19.2561875	19.76487564	19.55778228	19.2558391	19.17330095]
[19.34384653	19.09547582	19.44525251	19.11230083	19.21657532	19.16835263	19.55558132	19.32848754	19.16429387	19.07947852]
[19.16385007	18.90422415	19.14668636	18.9617132	18.99040213	19.05436645	19.2347662	19.03855566	19.02346413	18.91942911]]



5.3 Cross Validation

K-fold CV on Auto Data in R

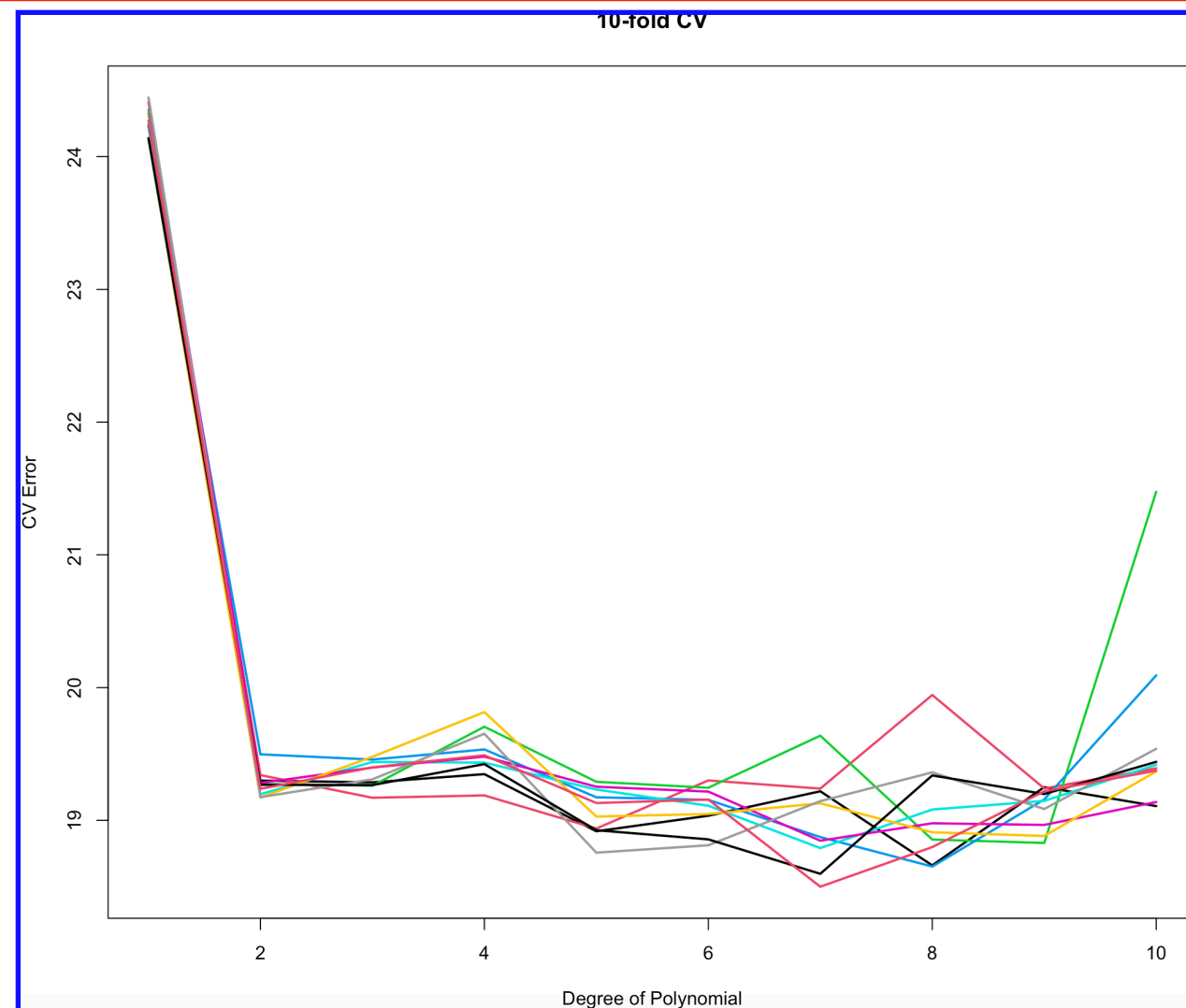
```
> #####  
> ##### K-fold Cross Validation  
> #####  
> ## Notice that the computation time is much shorter than that of LOOCV.  
> set.seed(17)  
> cv.error.10=rep(0,10)  
> for (i in 1:10){  
+   glm.fit=glm(mpg~poly(horsepower ,i),data=Auto)  
+   cv.error.10[i]=cv.glm(Auto,glm.fit,K=10)$delta[1]    ## K=10 means 10-fold cross validation  
+ }  
> cv.error.10    ## Based on the results, We still see little evidence that using cubic or higher-order polynomial terms leads to lower test error than simply using a quadratic fit.  
[1] 24.27207 19.26909 19.34805 19.29496 19.03198 18.89781 19.12061 19.14666 18.87013 20.95520  
> plot(1:length(cv.error.10),cv.error.10,type='b',col='blue',xlab='Degree of Polynomial',ylab='CV Error',main='10-fold CV')
```



5.3 Cross Validation

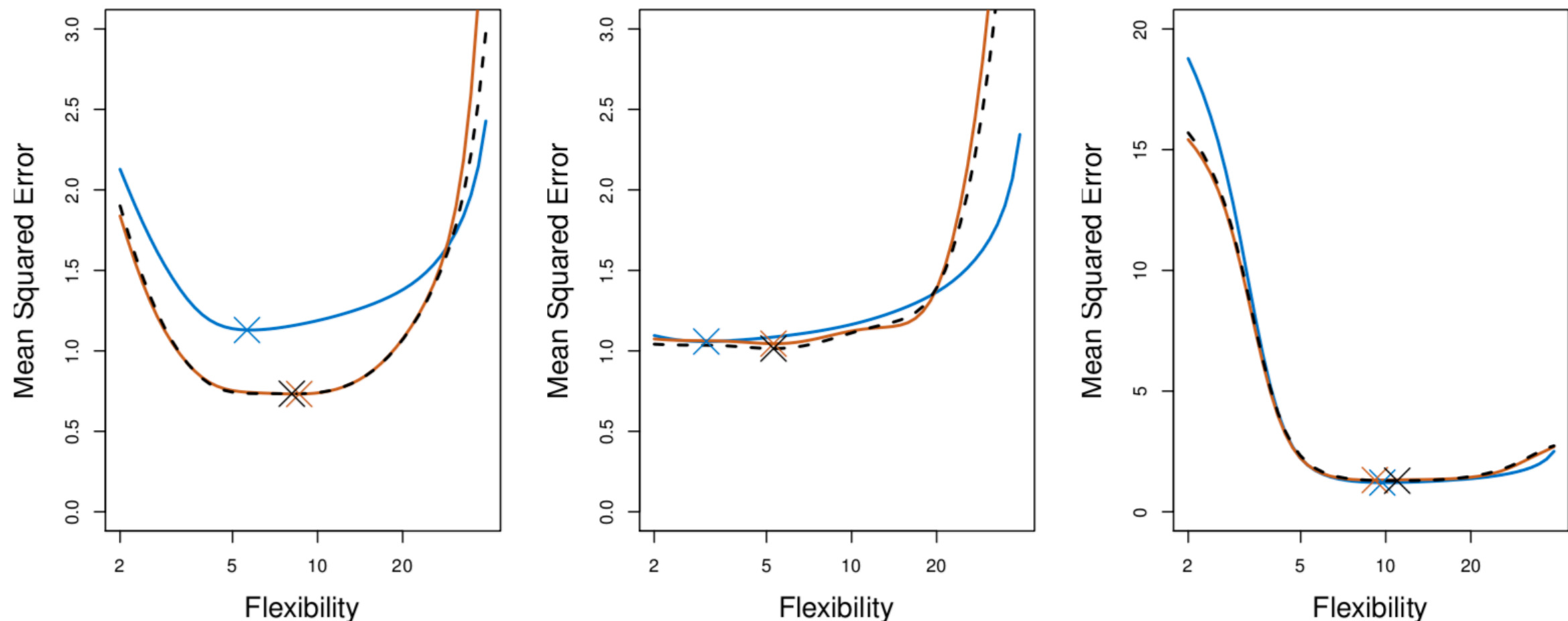
K-fold CV on Auto Data in R

```
#####
#####      K-fold Cross Validation
#####
## Notice that the computation time is much shorter than that of LOOCV.
cv.error.10=matrix(rep(0,100),10,10)
for(j in 1:10){
  set.seed(j+2020)
  for (i in 1:10){
    glm.fit=glm(mpg~poly(horsepower ,i),data=Auto)
    cv.error.10[i,j]=cv.glm(Auto,glm.fit,K=10)$delta[1]    ## K=10 means 10-fold cross validation
  }
}
matplot(cv.error.10, type = "l",col = 1:10, lty = 1, lwd = 2,xlab="Degree of Polynomial",ylab="CV Error",main="10-fold CV")
```



5.3 Cross Validation

K-fold cross validation



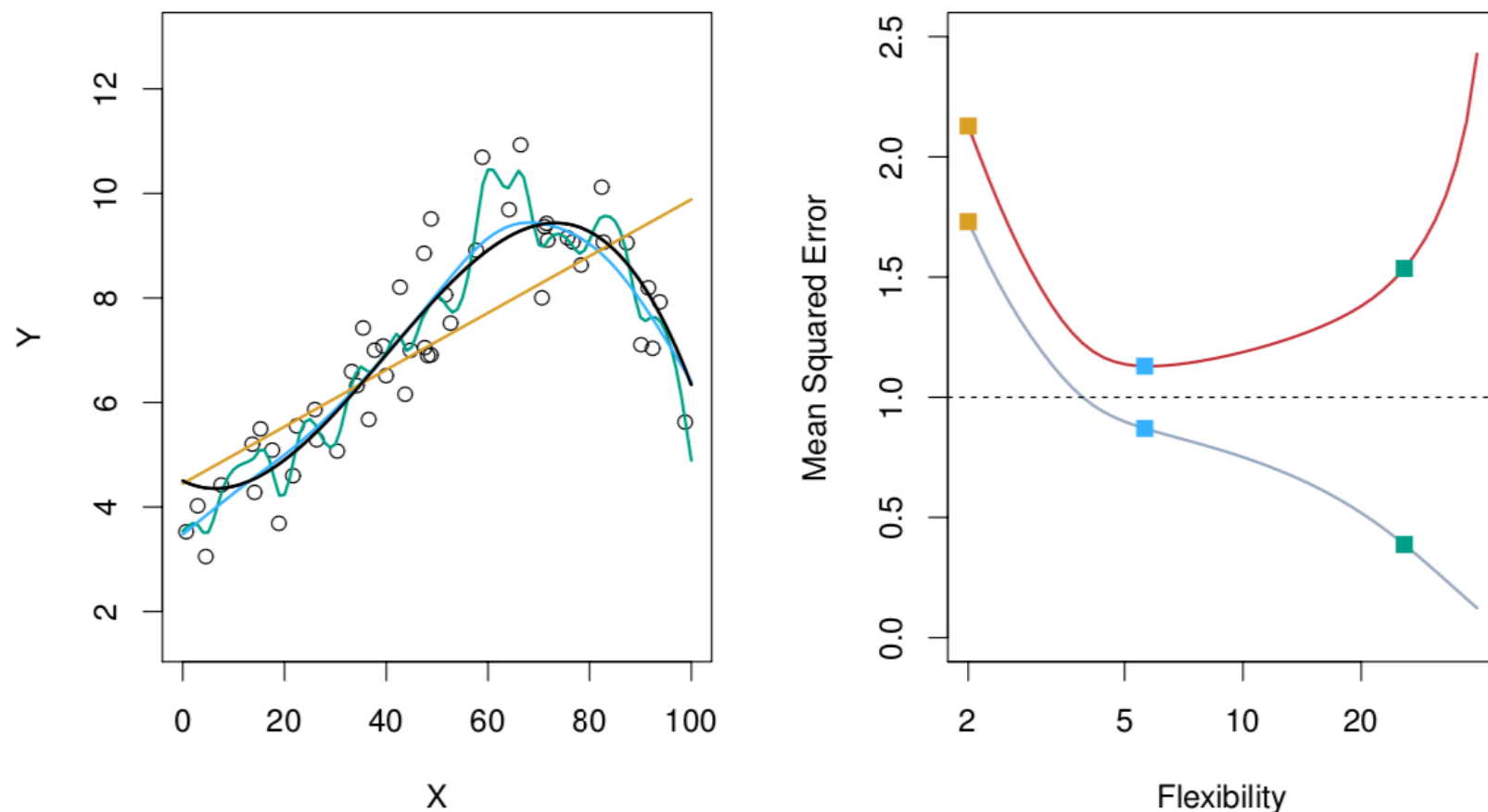
True and estimated test MSE for simulated data sets in **Scenario 1** (left), **Scenario 2** (center), and **Scenario 3** (right). The true test MSE is shown in blue, the LOOCV estimate is shown as a black dashed line, and the 10-fold CV estimate is shown in orange. The crosses indicate the minimum of each of the MSE curves.



next slides

5.3 Cross Validation

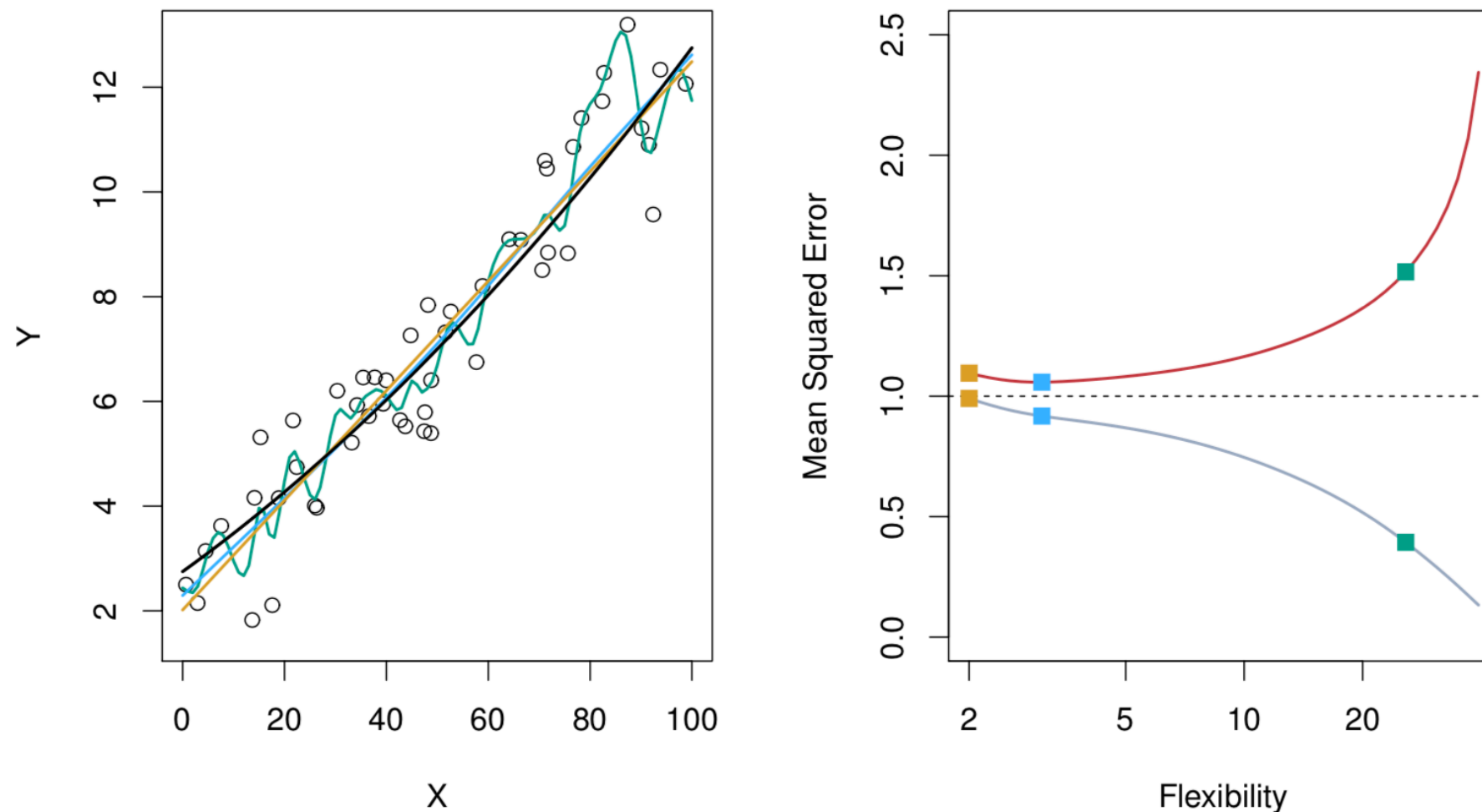
K-fold cross validation



Scenario 1 Left: Data simulated from f , shown in black. Three estimates of f are shown: the *linear regression line* (orange curve), and two smoothing spline fits (blue and green curves). Right: Training MSE (grey curve), test MSE (red curve), and minimum possible test MSE over all methods (dashed line). Squares represent the training and test MSEs for the three fits shown in the left-hand panel.

5.3 Cross Validation

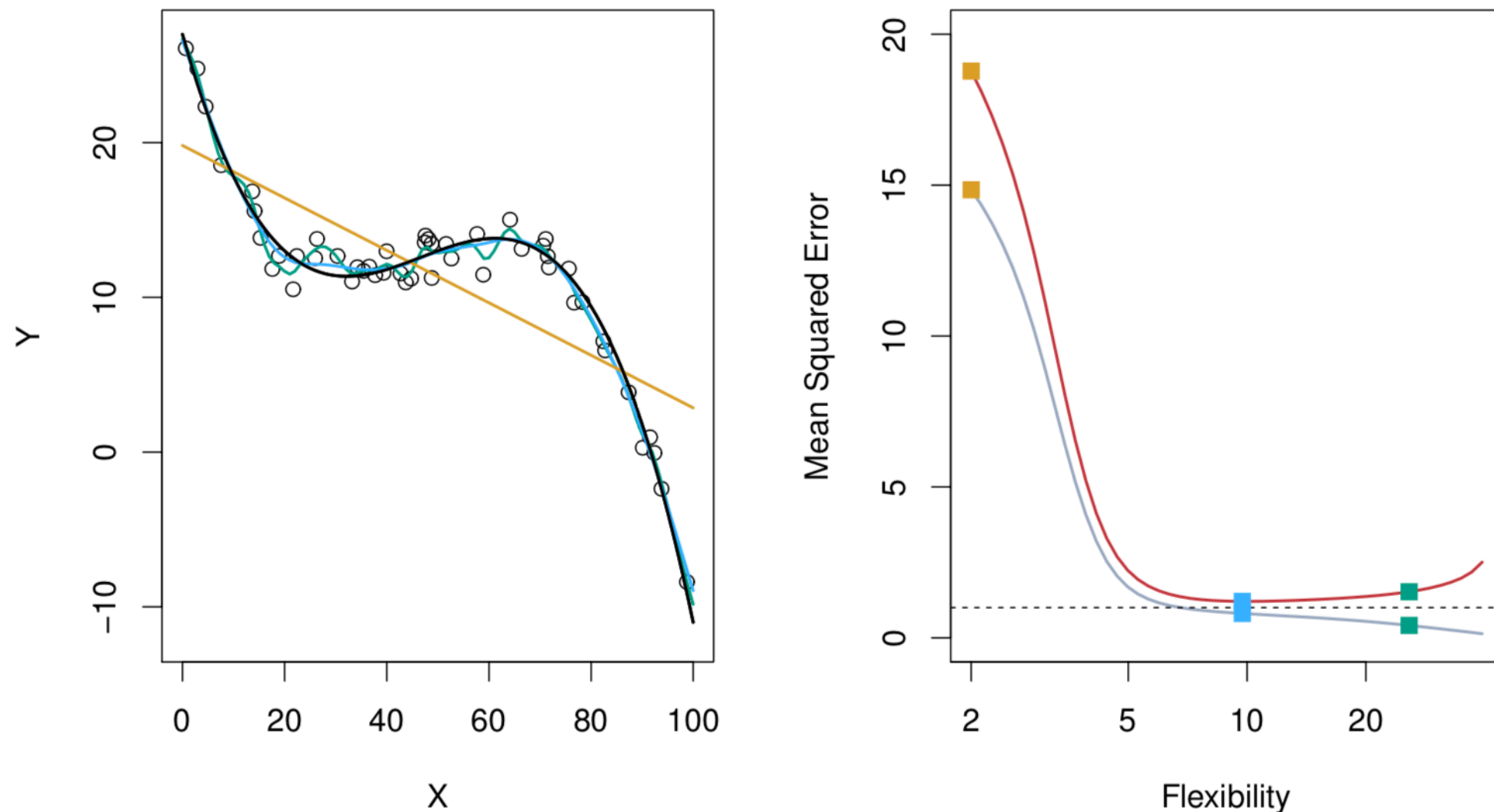
K-fold cross validation



Scenario 2 Details are as in **Scenario 1**, using a different true f that is much closer to linear. In this setting, linear regression provides a very good fit to the data.

5.3 Cross Validation

K-fold cross validation



Scenario 3 Details are as in **Scenario 1**, using a different f that is far from linear. In this setting, linear regression provides a very poor fit to the data.

5.3 Cross Validation

Complexity parameter at minimum test error

▶ *A family of models indexed by a parameter, usually representing flexibility or complexity of models.*

▶ *Such parameter is often called tuning parameter; it could even be a number of variables.*

▶ *Example: Order of polynomials of horsepower in the Auto data example.*

▶ *Example: Penalization parameters in ridge, lasso, etc (to be addressed in the next chapter.)*

▶ *Intend to find the best model within this family, i.e., to find the value of this tuning parameter.*

▶ *Care less of the actual value of the test error.*

▶ *In the above simulated data, all of the CV curves come close to identify the correct level of flexibility.*

5.3 Cross Validation

Example by KNN Regression in python

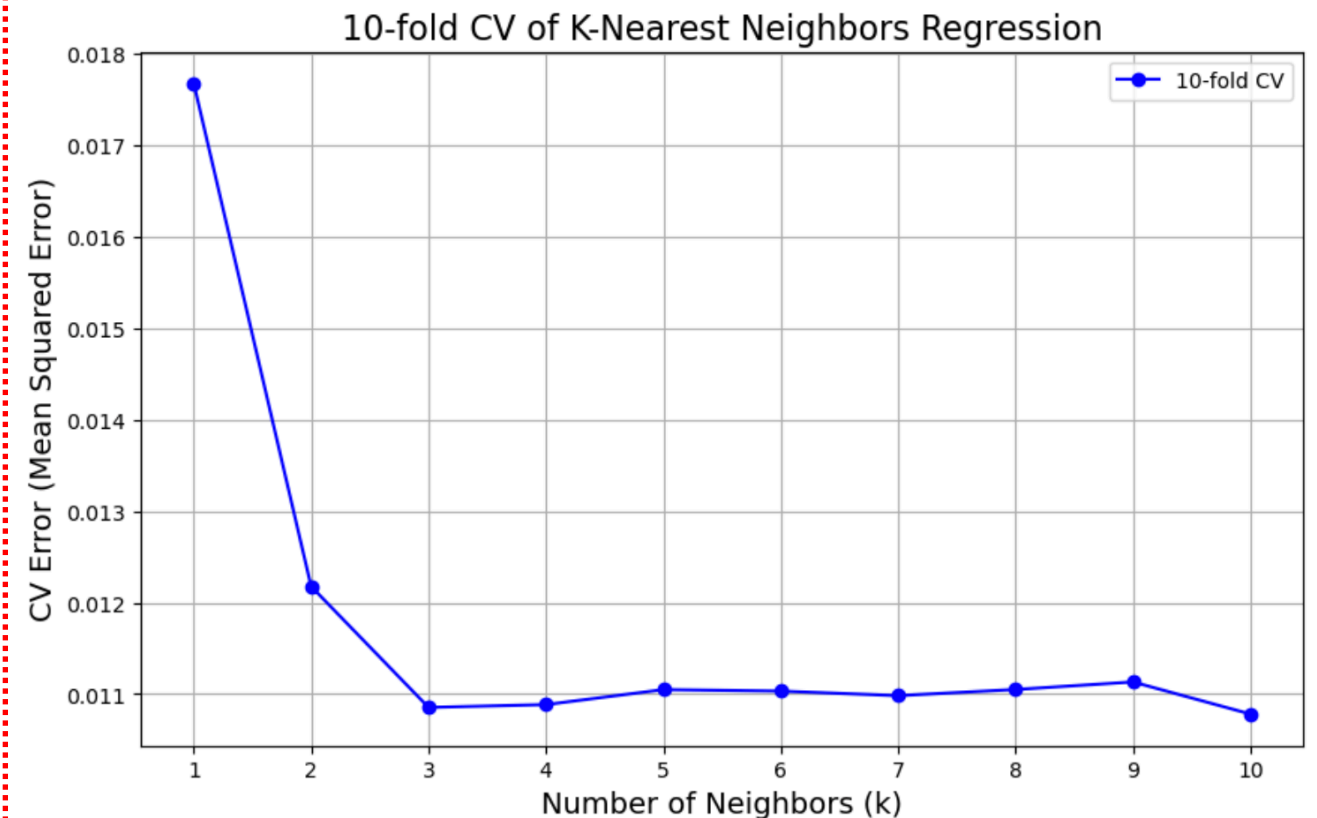
```
##### Using CV to determine the
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsRegressor
import matplotlib.pyplot as plt

# data generation
np.random.seed(17)
n_samples = 200
X = pd.DataFrame({'x': np.random.uniform(-1, 1, n_samples)})
noise = np.random.normal(0, 0.1, n_samples)
y = (X['x'] ** 2) + 2 + noise

###KNN with 10-CV
kf = KFold(n_splits=10, shuffle=True, random_state=17) # Fixed seed for reproducibility
k_values = range(1, 11)
CVERr = []
for k in k_values:
    err = 0 # Initialize error
    for train_index, test_index in kf.split(X):
        trainx, testx = X.iloc[train_index], X.iloc[test_index]
        trainy, testy = y.iloc[train_index], y.iloc[test_index]
        knnmod = KNeighborsRegressor(n_neighbors=k)
        knnmod.fit(trainx, trainy)
        test_y_pred = knnmod.predict(testx)
        squared_errors = (test_y_pred - testy) ** 2
        err += squared_errors.sum()
    cv_error = err / n_samples
    CVERr.append(cv_error)
    print(f"k = {k}: 10-fold CV MSE = {cv_error:.4f}")

###plot
plt.figure(figsize=(10, 6))
plt.plot(k_values, CVERr, marker='o', linestyle='-',
        color='blue', label='10-fold CV')
plt.title('10-fold CV of K-Nearest Neighbors Regression', fontsize=16)
plt.xlabel('Number of Neighbors (k)', fontsize=14)
plt.ylabel('CV Error (Mean Squared Error)', fontsize=14)
plt.xticks(k_values)
plt.grid(True)
plt.legend()
plt.show()
```

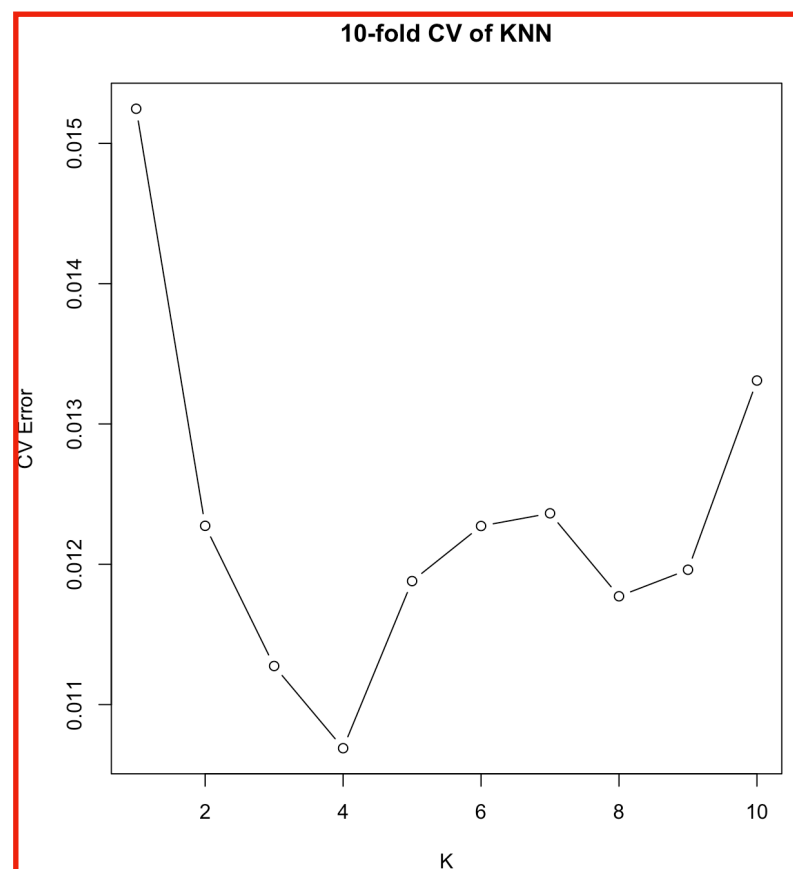
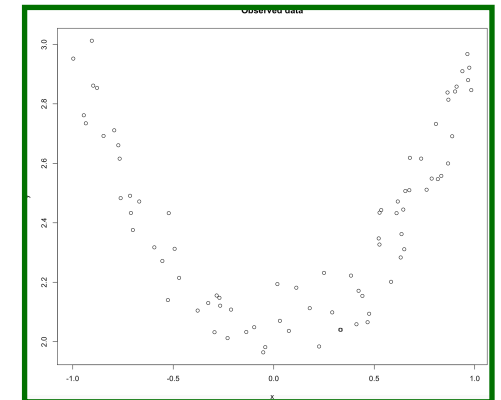
```
k = 1: 10-fold CV MSE = 0.0177
k = 2: 10-fold CV MSE = 0.0122
k = 3: 10-fold CV MSE = 0.0109
k = 4: 10-fold CV MSE = 0.0109
k = 5: 10-fold CV MSE = 0.0110
k = 6: 10-fold CV MSE = 0.0110
k = 7: 10-fold CV MSE = 0.0110
k = 8: 10-fold CV MSE = 0.0111
k = 9: 10-fold CV MSE = 0.0111
k = 10: 10-fold CV MSE = 0.0108
```



5.3 Cross Validation

Example by KNN Regression in R

```
##### Using CV to determine the best K in KNN
library(caret)
X<-data.frame(x=runif(80,min=-1,max=1))
y<-(X$x)^2+2+rnorm(40,mean=0,sd=0.1)
flds <- createFolds(1:80, k = 10, list = TRUE, returnTrain = FALSE)  ### random split into 10 folds
CVerErr<-rep(0,10)
for(k in 1:10){
  err<-0
  for(fold in flds){
    trainx<-data.frame(x=X[-fold,])
    testx<-data.frame(x=X[fold,])
    knnmod <- knnreg(trainx, y[-fold],k=k)
    test_y=predict(knnmod,testx)
    err<-err+sum((test_y-y[fold])^2)
  }
  CVerErr[k]<-err/80
}
plot(1:10,CVerErr,xlab="K",ylab="CV Error",main="10-fold CV of KNN")
```



5.3 Cross Validation

Bias and variance trade-off

- ▶ *In terms of bias of estimation of test error: validation set approach has more bias due to smaller size of training data; LOOCV is nearly unbiased; K-fold (e.g, $K=5$ or 10) has intermediate bias.*
- ▶ *In view of bias, LOOCV is most preferred; and K-fold cross validation next.*
- ▶ *But, K-fold cross validation has smaller variance than that of LOOCV.*
- ▶ *The n training sets LOOCV are too similar to each other. As a result, the trained models are too positively correlated.*
- ▶ *The K training sets of K-fold cross validation are much less similar to each other.*
- ▶ *As a result, the K-fold cross validation generally has less variance than LOOCV.*

5.3 Cross Validation

Cross validation for classification

▶ *MSE is a popular criterion to measure prediction/estimation accuracy for regression.*

▶ *There are other criteria.*

▶ *For classification with qualitative response, a natural choice is: 1 for incorrect classification and 0 for correct classification.*

▶ For LOOCV, this leads to $Err_i = I(y_i \neq \hat{y}_i^{(i)})$, where $y_i^{(i)}$ is the classification of i -th observation based on model fitted not using i -th observation.

▶ Then

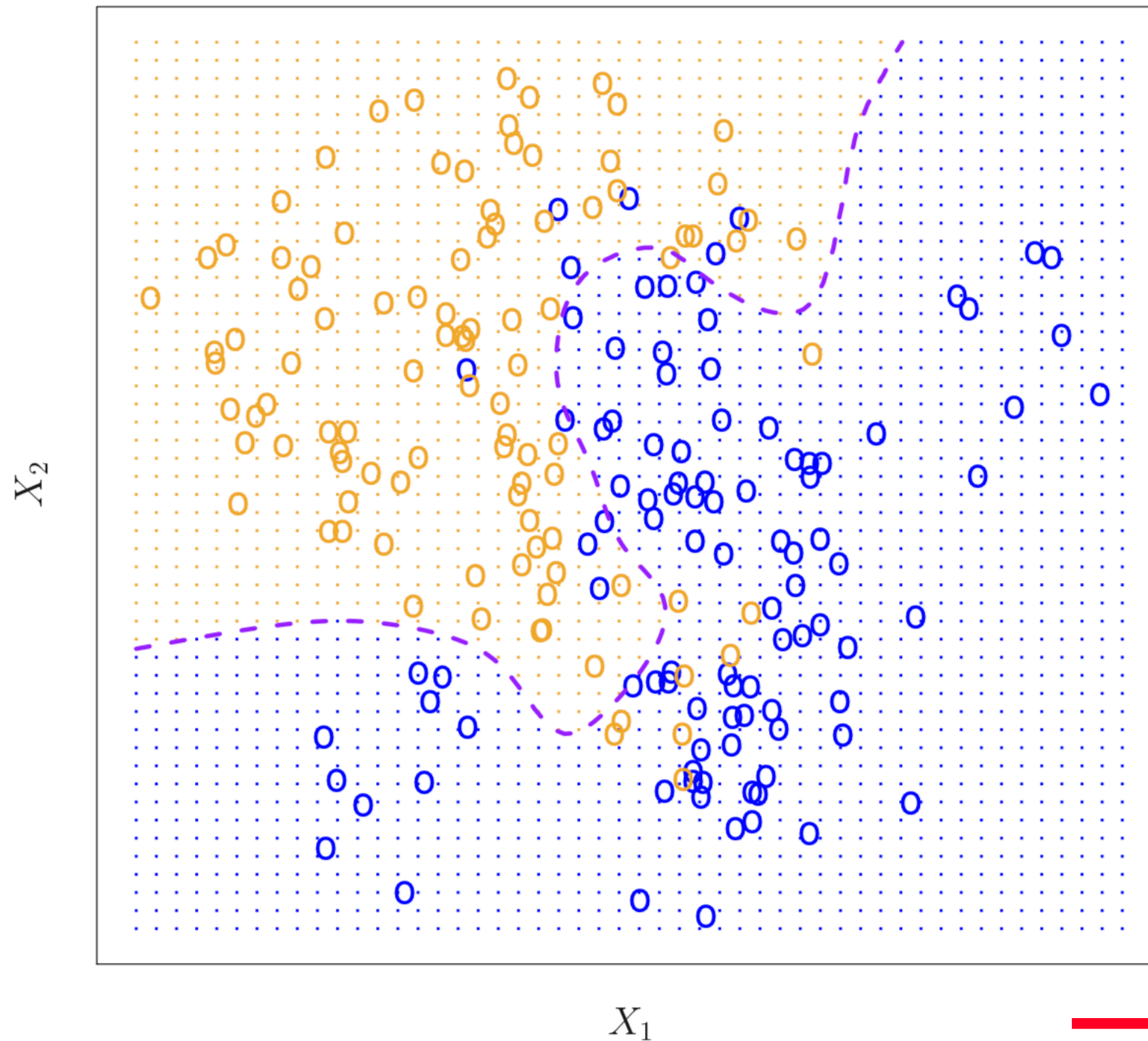
$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n Err_i$$

which is just the average number of incorrect classification.

5.3 Cross Validation

Cross validation for classification

Example



next slides

5.3 Cross Validation

Cross validation for classification

Example (cont.)

Figure (last slide):

A simulated data set consisting of 100 observations in each of two groups, indicated in blue and in orange.

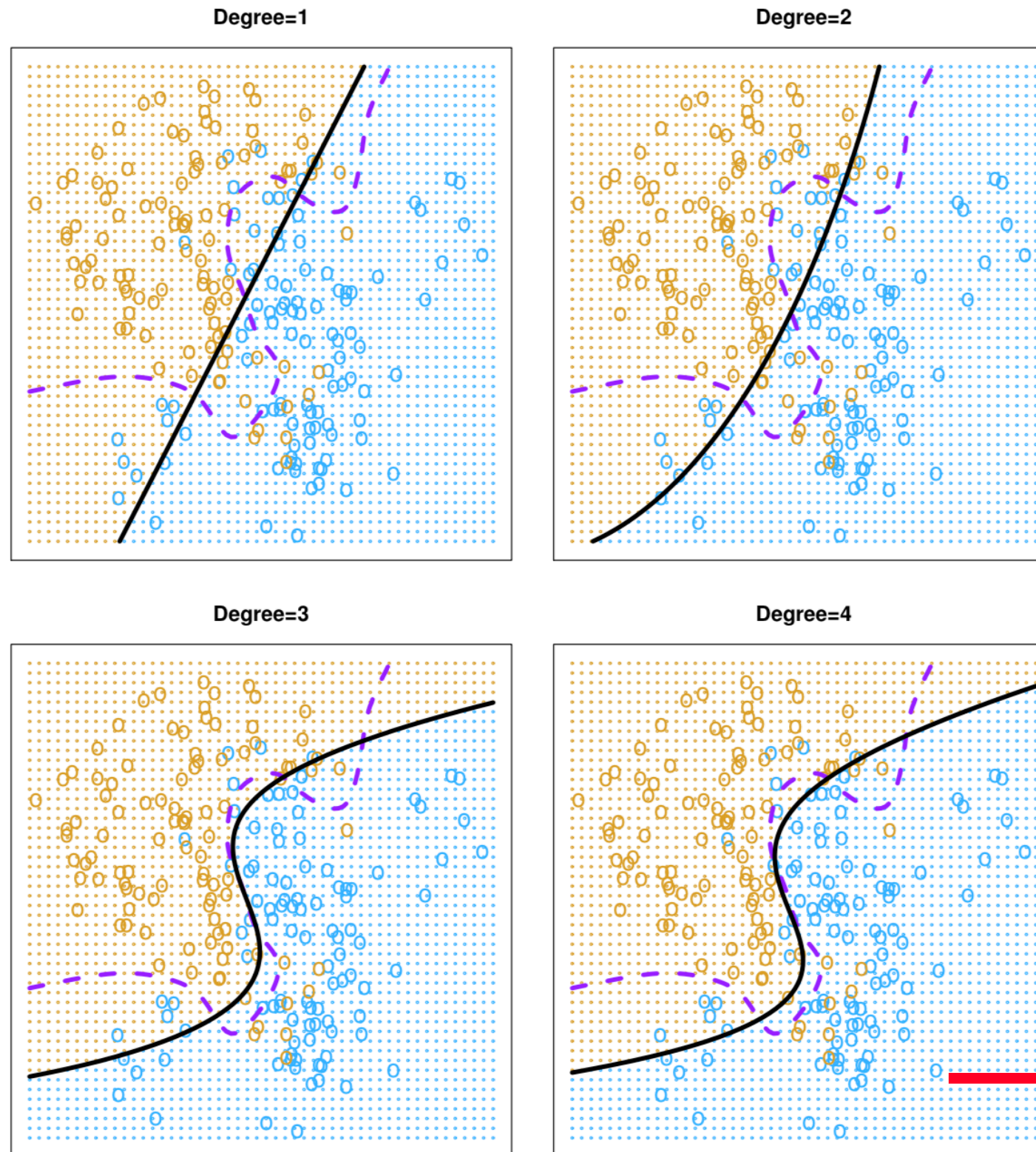
The purple dashed line represents the Bayes decision boundary.

The orange background grid indicates the region in which a test observation will be assigned to the orange class, and the blue background grid indicates the region in which a test observation will be assigned to the blue class.

5.3 Cross Validation

Cross validation for classification

Example (cont.)



5.3 Cross Validation

Cross validation for classification

Example (cont.)

Figure (last slide):

Logistic regression fits on the previous two-dimensional classification data.

The **Bayes decision boundary** is represented using a purple dashed line.

Estimated decision boundaries from **linear, quadratic, cubic and quartic** (degrees 1 to 4) logistic regressions are displayed in black.

The (TRUE) test error rates for the four logistic regression fits are respectively 0.201, 0.197, 0.160, and 0.162, while the Bayes error rate is 0.133.

Logistic polynomial regression:

$$P(Y = 1) = \frac{1}{1 + \exp(-\beta_0 - \beta_1 X - \beta_2 X^2 - \beta_3 X^3 - \dots - \beta_p X^p)}$$

5.3 Cross Validation

Cross validation for classification

Example (cont.)

 *last slide*

*The previous example is simulated.
The true population distribution is known.*

The figures 0.201, 0.197, 0.160, and 0.162, and 0.133 (for Bayes error rate) are the true test error, computed based on true population distribution.

In practice the true population distribution is unknown. Thus the true test error cannot be computed.

We use cross validation to solve the problem.

5.3 Cross Validation

Cross validation for classification

Example (cont.)

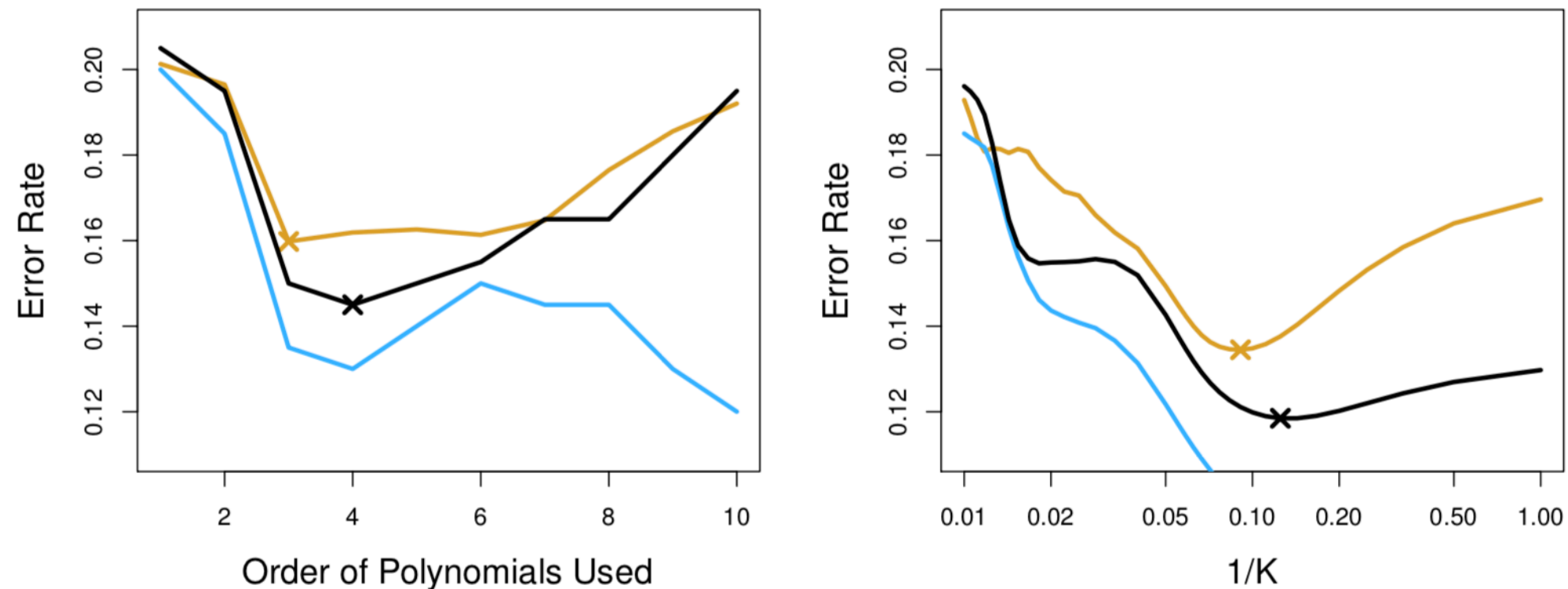


Figure: Test error (brown), training error (blue), and 10-fold CV error (black) on the previous two-dimensional classification data.

Left: Logistic regression using polynomial functions of the predictors. The order of the polynomials used is displayed on the x-axis.

Right: The KNN classifier with different values of K , the number of neighbors used in the KNN classifier.

5.3 Cross Validation

Cross validation for classification

- ▶ *Training error declines in general when model complexity increases.*
- ▶ *Training error some times even reaches 0.*
- ▶ *Test error generally declines first and then increases.*

5.3 Cross Validation

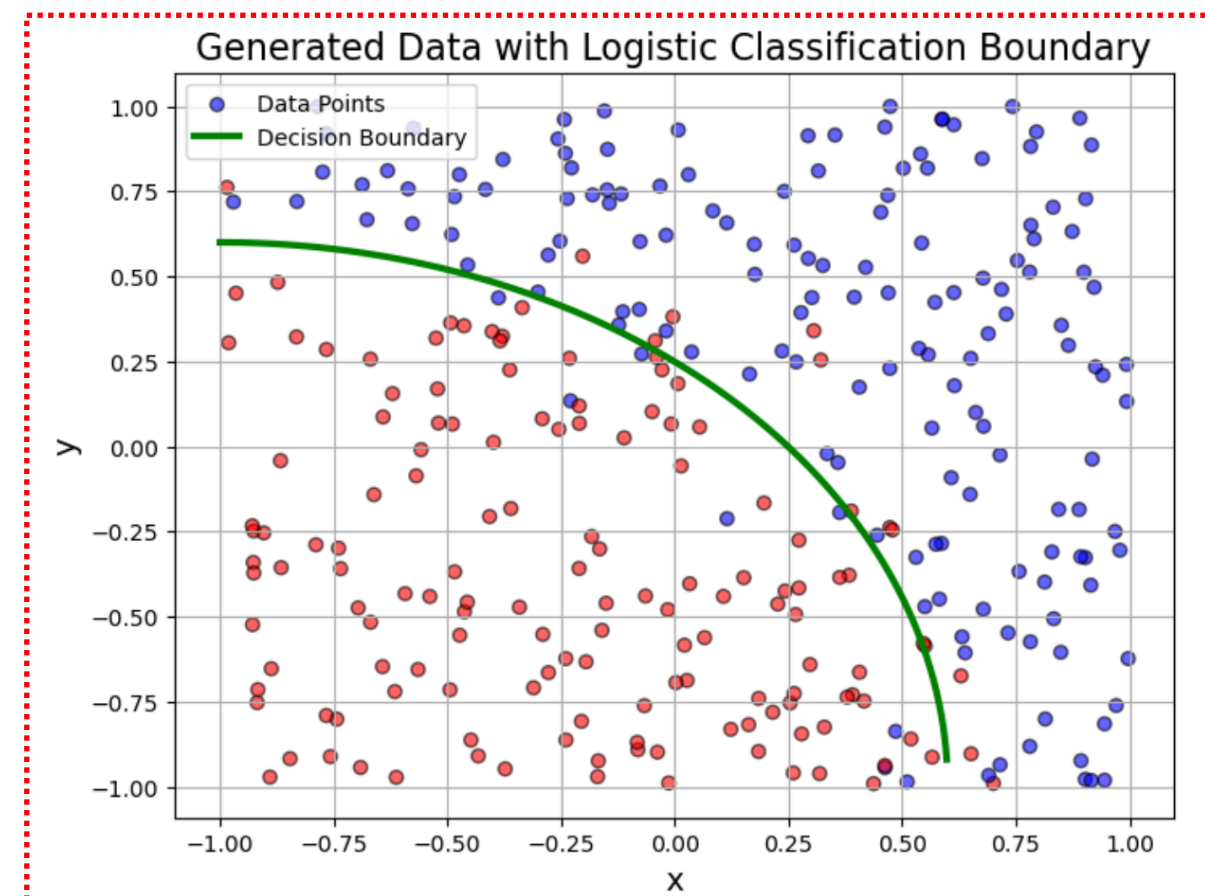
Cross validation for classification in Python

Logistic Polynomial Regression

```
#### CV for logistic regression
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import KFold, LeaveOneOut, cross_val_score
from sklearn.pipeline import make_pipeline
from mpl_toolkits.mplot3d import Axes3D # Optional for 3D plotting

# Set random seed for reproducibility
np.random.seed(20)
n_samples = 300
X = np.random.uniform(-1, 1, n_samples)
Y = np.random.uniform(-1, 1, n_samples)
noise = np.random.normal(0, 0.5, n_samples)
fakedat = pd.DataFrame({'x': X, 'y': Y, 'noise': noise})
# Define the class based on the circular boundary
# Class 1: (x+1)^2 + (y+1)^2 > 2.56
# Class 2: Otherwise
fakedat['class'] = np.where((fakedat['x'] + 1)**2 + (fakedat['y'] + 1)**2 > 2.56, 1, 2)
fakedat['class_str'] = fakedat['class'].astype(str)

### visualize the data
plt.figure(figsize=(8, 6))
colors = {1: 'red', 2: 'blue'}
plt.scatter(fakedat['x'], fakedat['y'], c=fakedat['class'], cmap='bwr', edgecolor='k', alpha=0.6, label='Data Points')
x_boundary = np.arange(-1, 0.6, 0.002)
mask = (x_boundary + 1)**2 <= 2.56
x_bnd = x_boundary[mask]
y_bnd = np.sqrt(2.56 - (x_bnd + 1)**2) - 1
plt.plot(x_bnd, y_bnd, color='green', linewidth=3, label='Decision Boundary')
plt.xlabel('x', fontsize=14)
plt.ylabel('y', fontsize=14)
plt.title('Generated Data with Logistic Classification Boundary', fontsize=16)
plt.legend()
plt.grid(True)
plt.show()
```



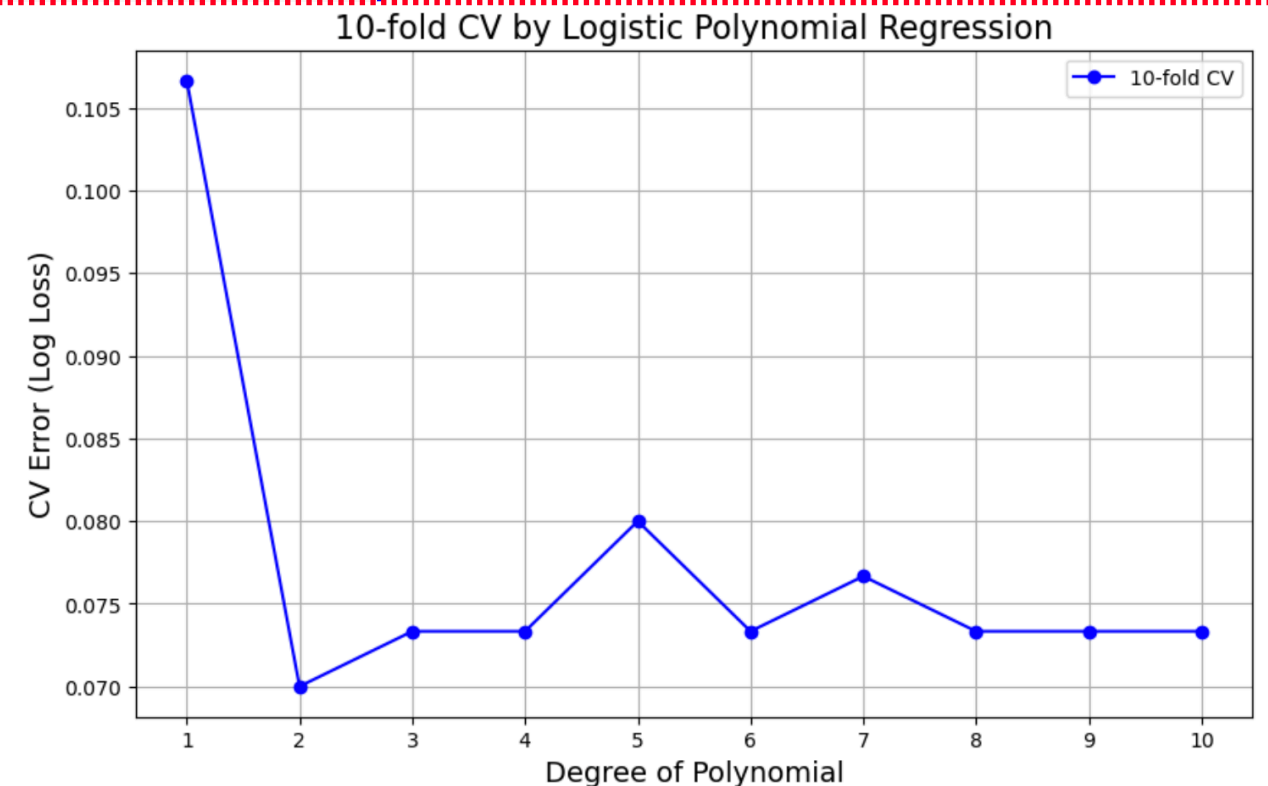
5.3 Cross Validation

Cross validation for classification in Python

Logistic Polynomial Regression

```
##### 10-CV logistic polynomial regression
# Map classes to binary labels: 1 -> 1, 2 -> 0
# This facilitates logistic regression in scikit-learn (binary classification: 1 vs 0)
fakedat['class_bin'] = fakedat['class'].map({1: 1, 2: 0})
kf = KFold(n_splits=10, shuffle=True, random_state=10)
cv_error_10 = []
print("Logistic Polynomial Regression using 10-Fold CV:")
for degree in range(1, 11):
    pipeline = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=True),
        LogisticRegression(solver='lbfgs', max_iter=1000)
    )
    #scores = cross_val_score(pipeline, fakedat[['x', 'y']], fakedat['class_bin'], cv=kf, scoring='neg_log_loss')## neg_log_likelihood loss
    #avg_log_loss = 1-scores.mean()
    scores = cross_val_score(pipeline, fakedat[['x', 'y']], fakedat['class_bin'], cv=kf, scoring='accuracy')## classification accuracy
    avg_log_loss = 1-scores.mean()
    cv_error_10.append(avg_log_loss)
    print(f"Degree {degree}: 10-fold CV Log Loss = {avg_log_loss:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), cv_error_10, marker='o', linestyle='-', color='blue', label='10-fold CV')
plt.title('10-fold CV by Logistic Polynomial Regression', fontsize=16)
plt.xlabel('Degree of Polynomial', fontsize=14)
plt.ylabel('CV Error (Log Loss)', fontsize=14)
plt.xticks(range(1, 11))
plt.grid(True)
plt.legend()
plt.show()
```



5.3 Cross Validation

How does K affect the bias and variance of the CV error?

5.3 Cross Validation

Does Cross Validation over-estimate or under-estimate the true test error?

5.4 Bootstrap

Assessing estimators' uncertainty

- ▶ Suppose we have data $x_1, \dots, x_n$, representing the ages of n randomly selected people in HK.
- ▶ Use sample mean $\bar{x}$ to estimate the population mean μ , the average age of all residents of HK.
- ▶ How to justify the estimation error $\bar{x} - \mu$? Usually by t -confidence interval, test of hypothesis.
- ▶ *They rely on normality assumption or central limit theorem.*
- ▶ *Is there another reliable way? **Just bootstrap***

5.4 Bootstrap

Approximate the distribution of $\bar{x} - \mu$

- ▶ Take n random sample (with replacement) from $x_1, \dots, x_n$.
- ▶ calculate the sample mean of the “re-sample”, denoted as $\bar{x}_1^*$.
- ▶ Repeat the above a large number M times. We have $\bar{x}_1^*, \bar{x}_2^*, \dots, \bar{x}_M^*$.
- ▶ Use the distribution of $\bar{x}_1^* - \bar{x}, \dots, \bar{x}_M^* - \bar{x}$ to approximate that of $\bar{x} - \mu$.

5.4 Bootstrap

Rationale of bootstrap

▶ *Essential idea: Treat the data distribution (more professionally called empirical distribution) as a proxy of the population distribution.*

▶ *Mimic the data generation from the true population, by trying resampling from the empirical distribution.*

▶ Mimic your statistical procedure (such as computing an estimate $\bar{x}$) on data, by doing the same on the resampled data.

▶ *Evaluate your statistical procedure (which may be difficult because it involves randomness and the unknown population distribution) by evaluating your analogue procedures on the re-samples.*

5.4 Bootstrap

Example

▶ X and Y are two random variables. Then minimizer of $\text{var}(\alpha X + (1 - \alpha)Y)$ is

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}$$

▶ Data: $(X_1, Y_1), \dots, (X_n, Y_n)$.

▶ We can compute sample variances and covariances.

▶ Estimate α by $\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}$

▶ How to evaluate $\hat{\alpha} - \alpha$, (remember $\hat{\alpha}$ is random and α is unknown).

▶ Use Bootstrap

5.4 Bootstrap

Example (cont.)

Sample n resamples from $(X_1, Y_1), \dots, (X_n, Y_n)$, and compute the sample the sample variance and covariances for this resample. And then compute

$$\hat{\alpha}^* = \frac{(\hat{\sigma}_Y^*)^2 - \hat{\sigma}_{XY}^*}{(\hat{\sigma}_X^*)^2 + (\hat{\sigma}_Y^*)^2 - 2\hat{\sigma}_{XY}^*}$$

Repeat this procedure, and we have $\hat{\alpha}_1^*, \dots, \hat{\alpha}_M^*$ for a large M .

Use the distribution of $\hat{\alpha}_1^* - \hat{\alpha}, \dots, \hat{\alpha}_M^* - \hat{\alpha}$ to approximate the distribution of $\hat{\alpha} - \alpha$.

For example, we can use $\frac{1}{M} \sum_{j=1}^M (\hat{\alpha}_j^* - \hat{\alpha})^2$ to estimate $E(\hat{\alpha} - \alpha)^2$.

5.4 Bootstrap

Example (cont.)

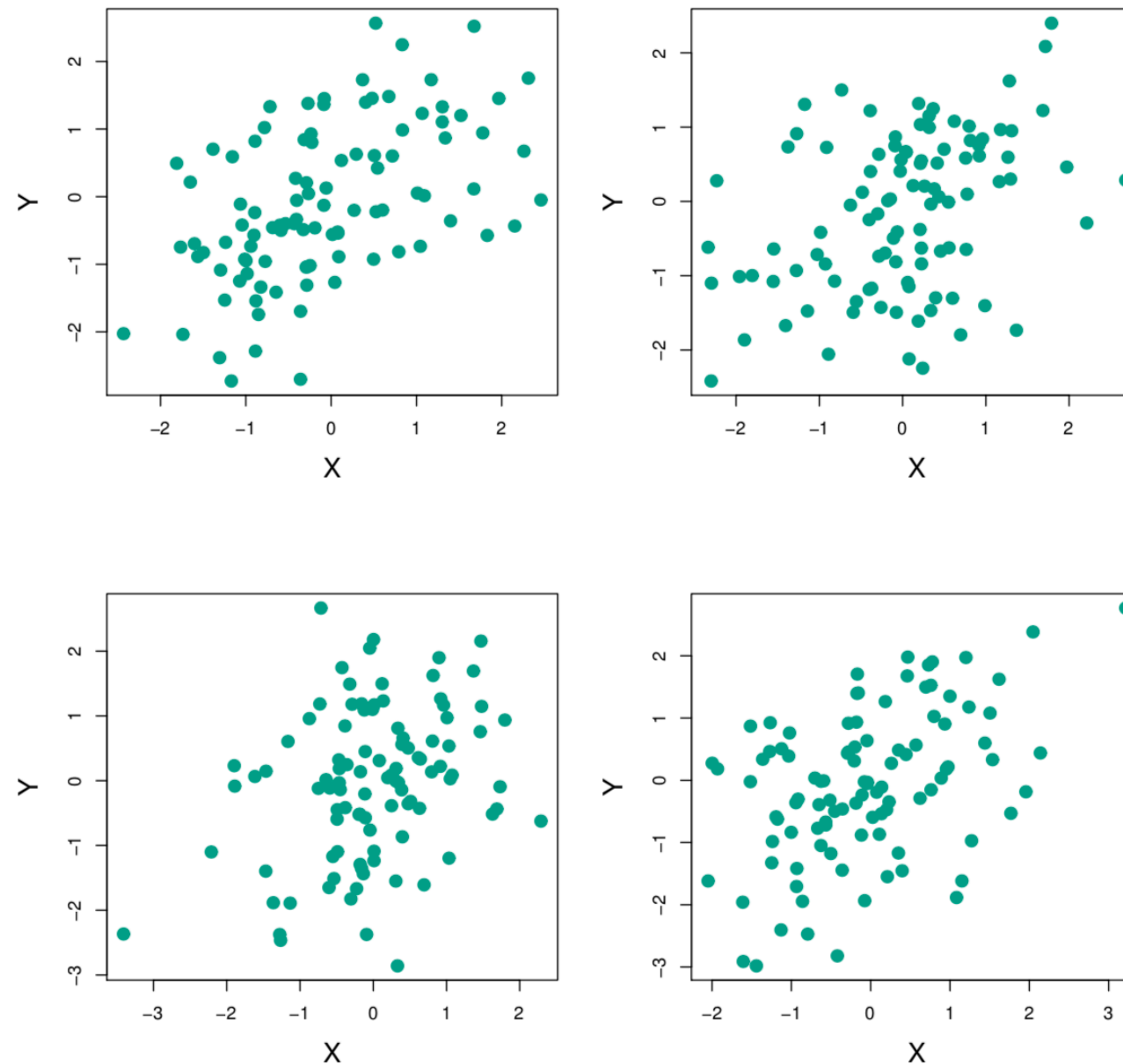


Figure: Each panel displays 100 simulated returns for investments X and Y . From left to right and top to bottom, the resulting estimates for α are 0.576, 0.532, 0.657, and 0.651.

5.4 Bootstrap

Example (cont.)

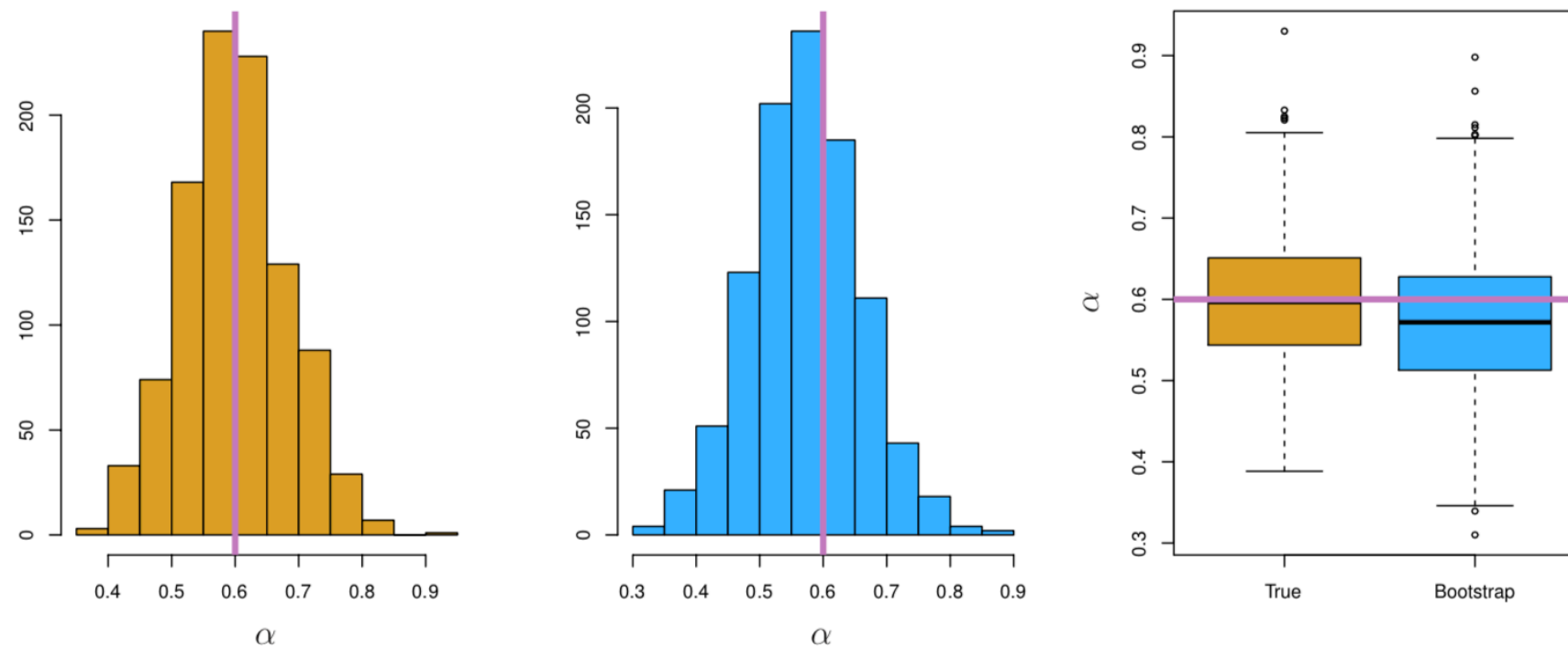


Figure: Left: A histogram of the estimates of α obtained by generating 1,000 simulated data sets from the true population. Center: A histogram of the estimates of α obtained from 1,000 bootstrap samples from a single data set. Right: The estimates of α displayed in the left and center panels are shown as boxplots. In each panel, the pink line indicates the true value of α .

5.4 Bootstrap

Example (cont.)

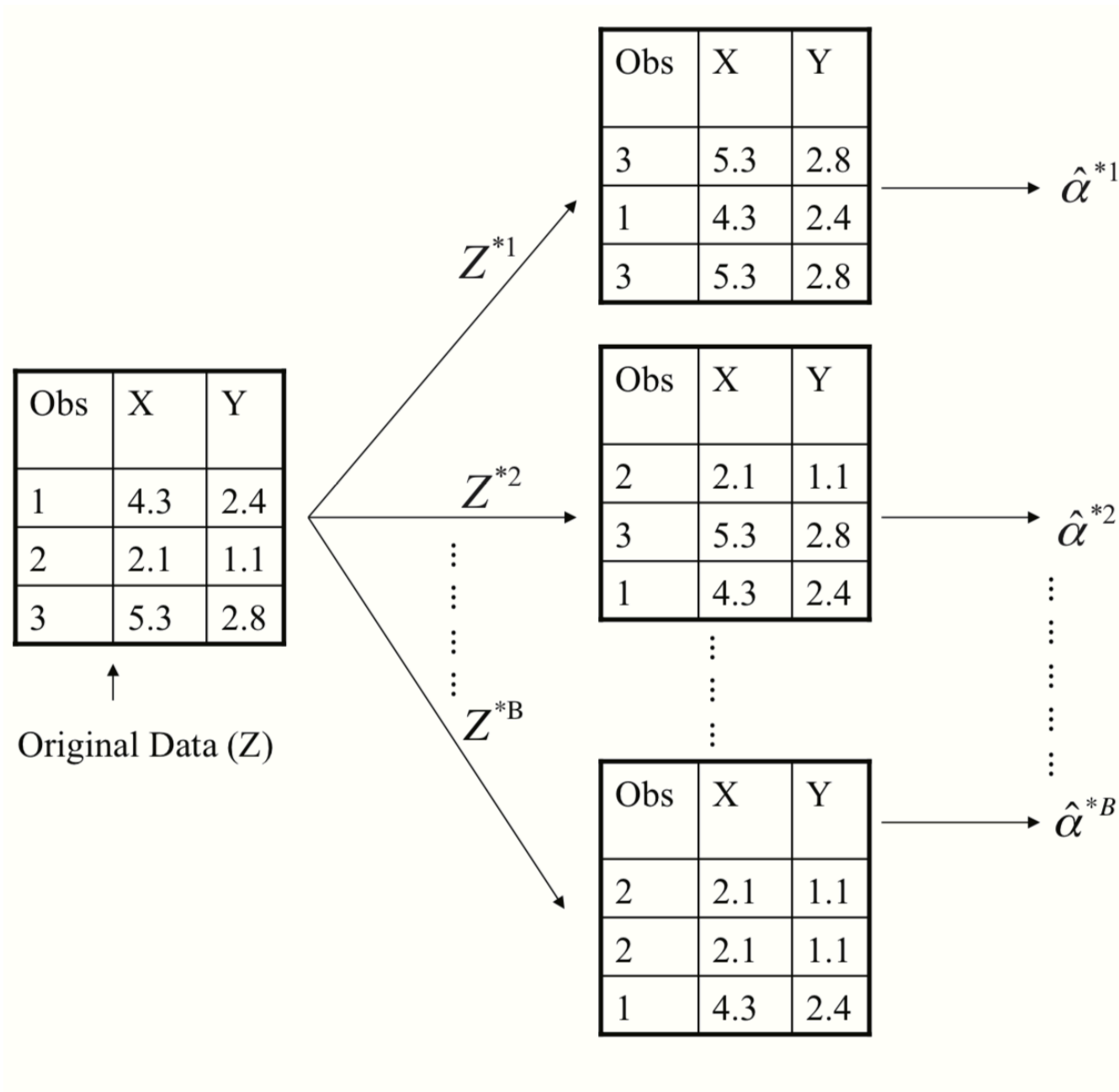


Figure:

A graphical illustration of the bootstrap approach on a small sample containing $n = 3$ observations.

Each bootstrap data set contains n observations, sampled with replacement from the original data set.

Each bootstrap data set is used to obtain an estimate.

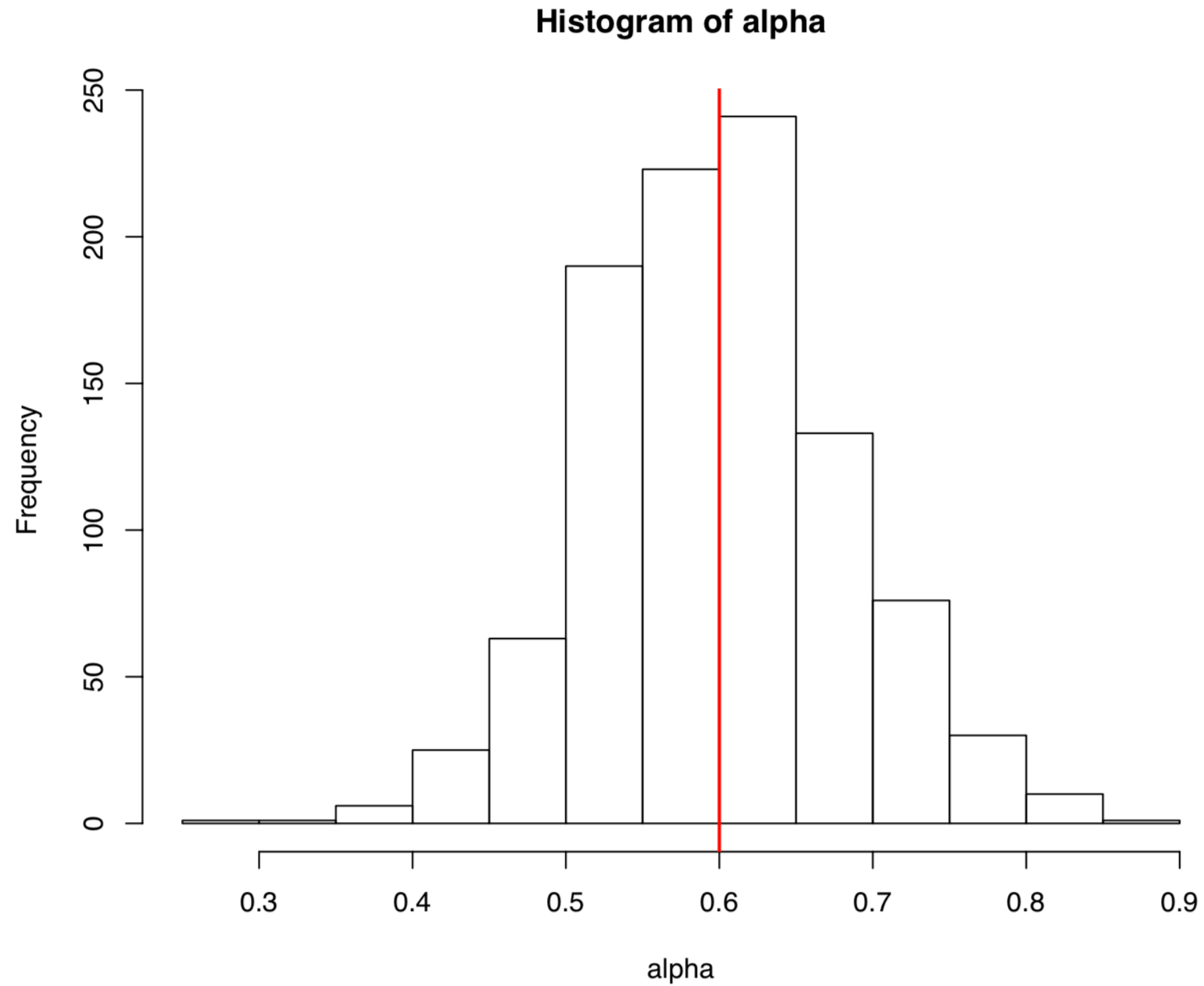
5.4 Bootstrap

The R code

```
library(mvtnorm)
set.seed(1)
n <- 100
m <- 1000
sigmaX <- 1
sigmaY <- 1.25
sigmaXY <- 0.5
sigMat <- matrix(c(sigmaX,sigmaXY,sigmaXY,sigmaY),2,2)

alpha <- rep(0,m)
for(i in 1:m){
  returns <- rmvnorm(n,rep(0,2),sigMat)
  X <- returns[,1]
  Y <- returns[,2]
  alpha[i] <- (var(Y)-cov(X,Y))/(var(X)+var(Y)-2*cov(X,Y))
}
mean(alpha)
sd(alpha)
hist(alpha,10)
abline(v=0.6,col="red",lwd=2)
```

5.4 Bootstrap



5.4 Bootstrap

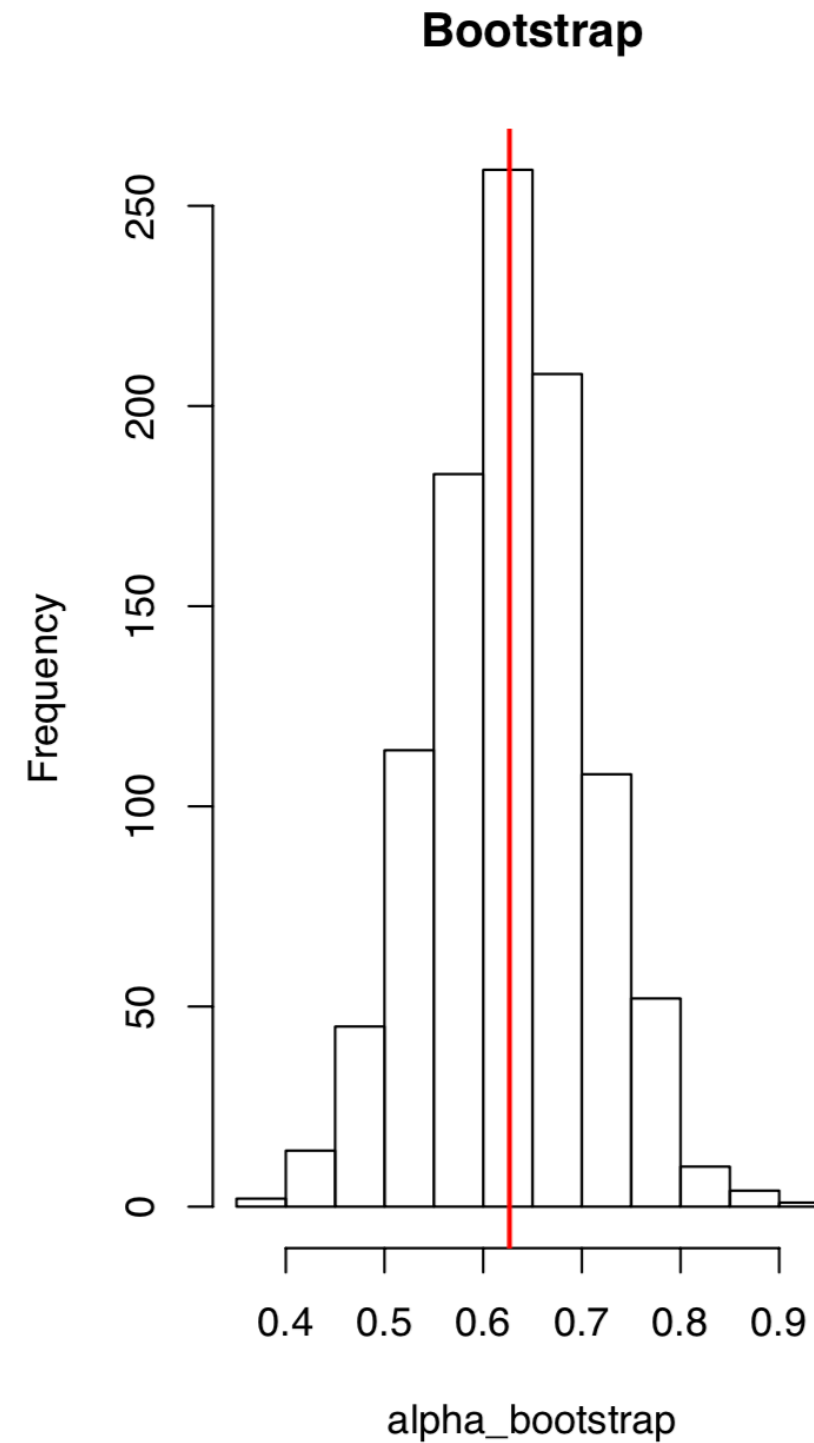
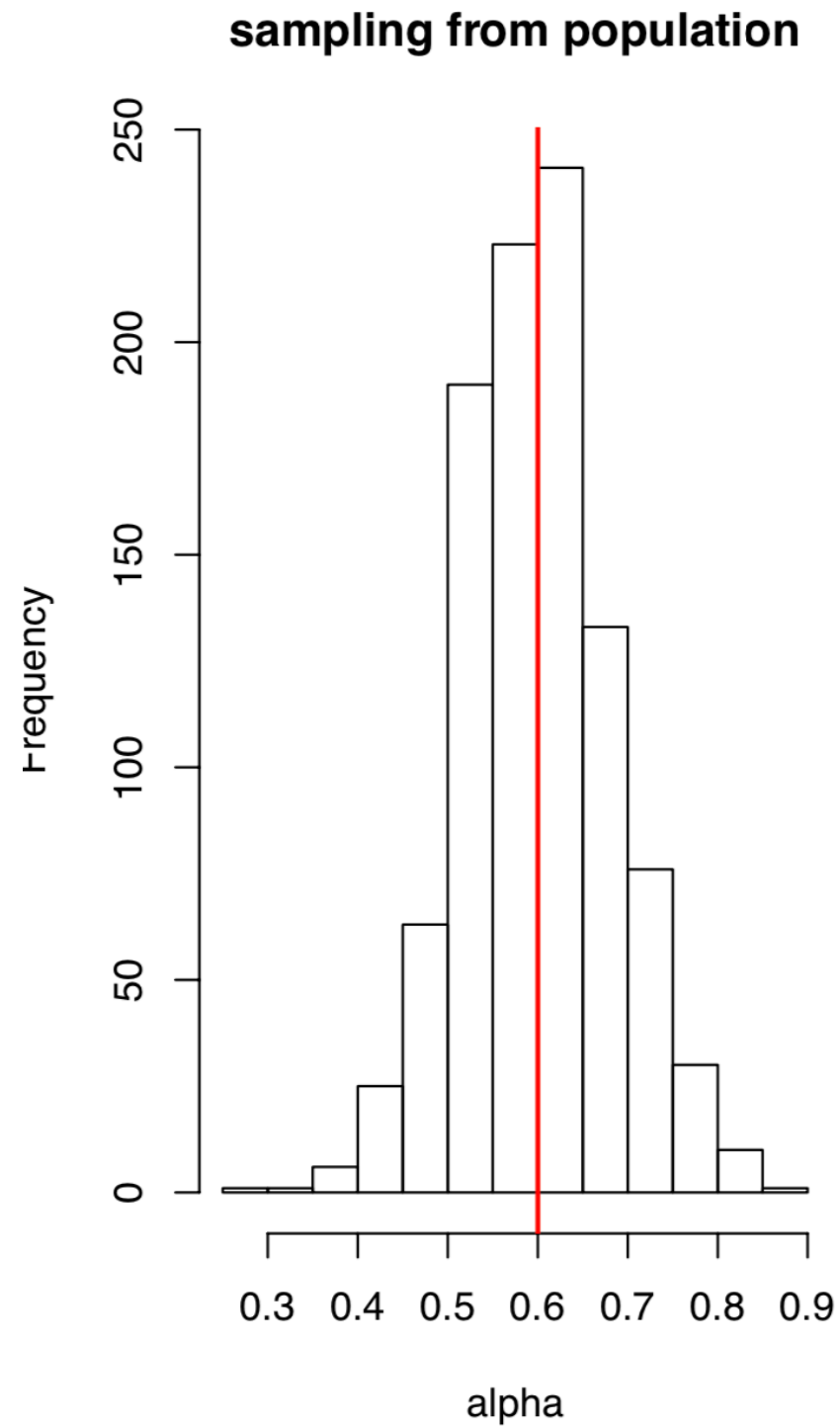
The R code

```
set.seed(1)
n <- 100
B <- 1000
returns <- rmvnorm(n,rep(0,2),sigMat) # the dataset is fixed once sampled

alpha_bootstrap <- rep(0,B)
for(i in 1:B){
  # sample with replacement
  returns_i <- returns[sample(1:nrow(returns),n,replace = T),]
  X <- returns_i[,1]
  Y <- returns_i[,2]
  alpha_bootstrap[i] <- (var(Y)-cov(X,Y))/(var(X)+var(Y)-2*cov(X,Y))
}

library(repr)
options(repr.plot.width=8, repr.plot.height=4)
par(mfrow=c(1,2))
hist(alpha,10,main = "sampling from population")
abline(v=0.6,col="red",lwd=2)
mean(alpha_bootstrap)
sd(alpha_bootstrap)
hist(alpha_bootstrap,10,main="Bootstrap")
abline(v=mean(alpha_bootstrap),col="red",lwd=2)
```

5.4 Bootstrap



5.4 Bootstrap

Example Using R

Use Bootstrap to Estimate the Accuracy of Linear Regression

```
> #####  
> ##### Use Bootstrap to Estimate the Accuracy of a Linear Regression Model  
> #####  
>  
> boot.fn=function(data,index){                                ## create the function for bootstrap to use  
+   return(coef(lm(mpg~horsepower ,data=data,subset=index)))}  
> boot.fn(Auto, 1:392)  
(Intercept) horsepower  
39.9358610 -0.1578447
```

```
> set.seed(1)  
> boot.fn(Auto,sample(392,392,replace=T))                        ## linear regression on one bootstrap sample  
(Intercept) horsepower  
40.3404517 -0.1634868  
> boot.fn(Auto,sample(392,392,replace=T))                        ## linear regression on another bootstrap sample  
(Intercept) horsepower  
40.1186906 -0.1577063
```

5.4 Bootstrap

KNN regression with Bootstrap in python

```
##### confidence interval of KNN regression by Bootstrap
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.utils import resample
import matplotlib.pyplot as plt
import seaborn as sns

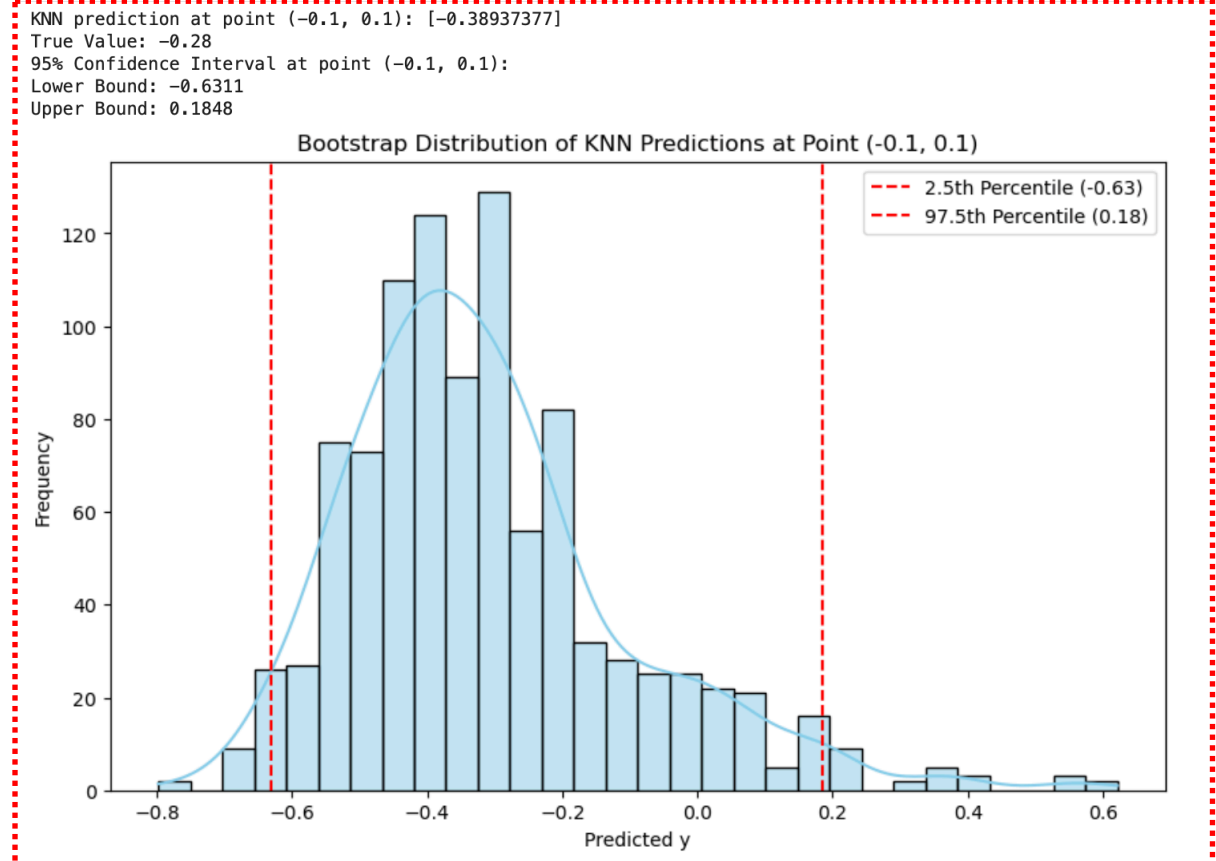
# Set random seed for reproducibility
np.random.seed(42)
n_samples = 500
x1 = np.random.uniform(-1, 1, n_samples)
x2 = np.random.uniform(-1, 1, n_samples)
y = 3 * x1 + 2 * x2**2 + np.random.normal(0, 0.5, n_samples)
data = pd.DataFrame({
    'x1': x1,
    'x2': x2,
    'y': y
})

predict_point = np.array([[-0.1, 0.1]]) # (x1, x2) = (0, 0.1)
knn = KNeighborsRegressor(n_neighbors=knn_k)
knn.fit(data[['x1', 'x2']].values, data[['y']].values)
knnpred=knn.predict(predict_point)[0]
print(f"KNN prediction at point {tuple(predict_point[0])}: {knnpred}")
print(f"True Value: -0.28")

n_bootstraps = 1000 # Number of bootstrap samples
knn_k = 5 # Number of neighbors for KNN
bootstrap_preds = []
for i in range(n_bootstraps):
    bootstrap_sample = resample(data, replace=True, n_samples=n_samples)
    X_boot = bootstrap_sample[['x1', 'x2']].values
    y_boot = bootstrap_sample[['y']].values
    knn = KNeighborsRegressor(n_neighbors=knn_k)
    knn.fit(X_boot, y_boot)
    pred = knn.predict(predict_point)[0]
    bootstrap_preds.append(pred)

bootstrap_preds = np.array(bootstrap_preds)
lower_bound = np.percentile(bootstrap_preds, 2.5)
upper_bound = np.percentile(bootstrap_preds, 97.5)
confidence_interval = (lower_bound, upper_bound)
print(f"95% Confidence Interval at point {tuple(predict_point[0])}:")
print(f"Lower Bound: {lower_bound:.4f}")
print(f"Upper Bound: {upper_bound:.4f}")

# Plot the Bootstrap Distribution
plt.figure(figsize=(10, 6))
sns.histplot(bootstrap_preds, bins=30, kde=True, color='skyblue', edgecolor='black')
plt.axvline(lower_bound, color='red', linestyle='--', label=f'2.5th Percentile ({lower_bound:.2f})')
plt.axvline(upper_bound, color='red', linestyle='--', label=f'97.5th Percentile ({upper_bound:.2f})')
plt.title(f'Bootstrap Distribution of KNN Predictions at Point {tuple(predict_point[0])}')
plt.xlabel('Predicted y')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



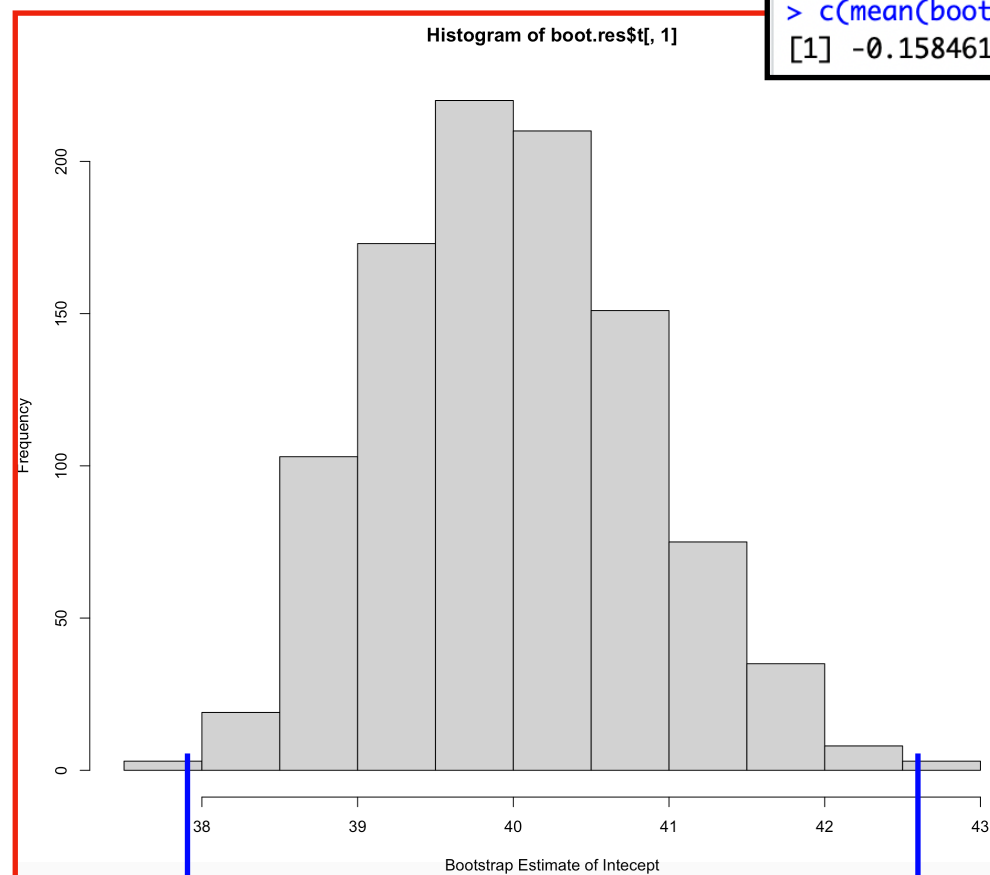
5.4 Bootstrap

Example Using R

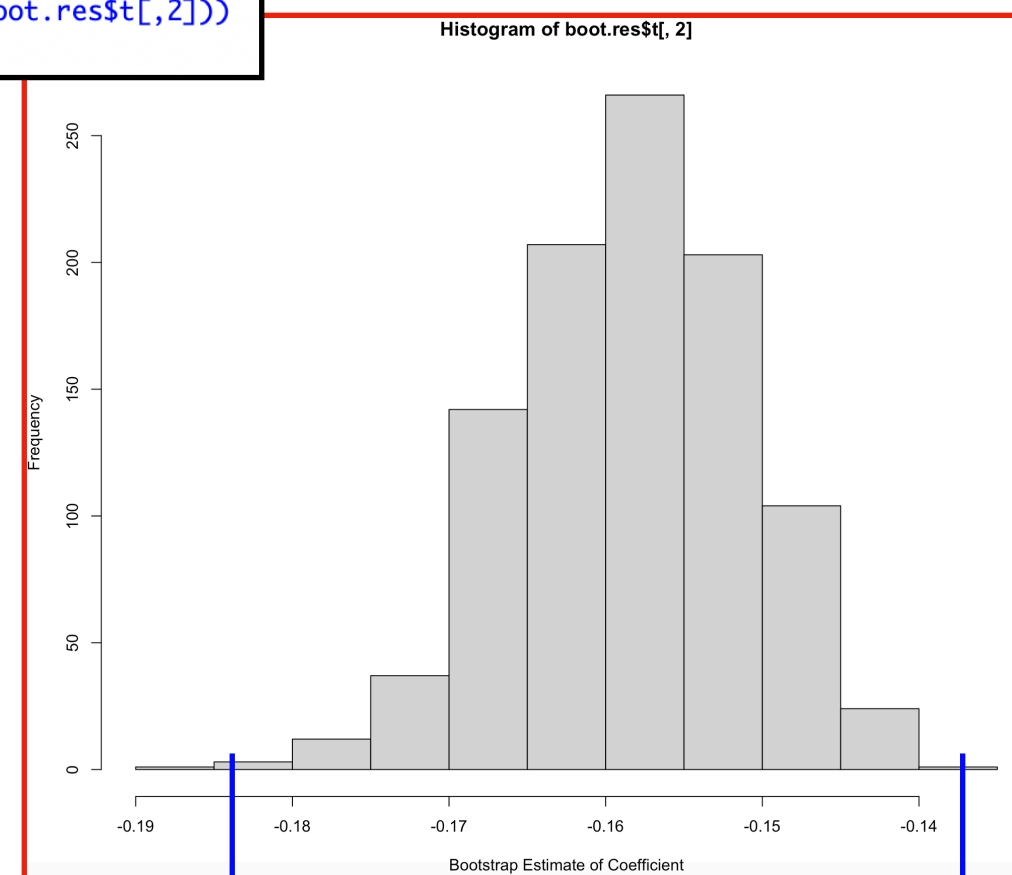
Use Bootstrap to Estimate the Accuracy of Linear Regression

```
> boot.res<-boot(Auto ,boot.fn ,1000)                                ## output the the bootstrap statistics based on 1000 bootstrap samples
> hist(boot.res$t[,1],xlab="Bootstrap Estimate of Intecept")
> hist(boot.res$t[,2],xlab="Bootstrap Estimate of Coefficient")
>
> summary(lm(mpg~horsepower ,data=Auto))$coef                      ## the summary of estimated std.err by linear regression method, compare it to bootstrap estimate
      Estimate Std. Error  t value    Pr(>|t|)
(Intercept) 39.9358610 0.717498656 55.65984 1.220362e-187
horsepower  -0.1578447 0.006445501 -24.48914 7.031989e-81
```

```
> c(mean(boot.res$t[,1]),sd(boot.res$t[,1]))
[1] 39.9903123 0.8412898
> c(mean(boot.res$t[,2]),sd(boot.res$t[,2]))
[1] -0.158461823 0.007343073
```



the interval consisting of 95% estimates is the 95% Bootstrap confidence interval



the interval consisting of 95% estimates is the 95% Bootstrap confidence interval

5.4 Bootstrap

Example Using R

Use Bootstrap to Estimate the Accuracy of Linear Regression

```
> ### Below we compute the bootstrap standard error estimates and the standard linear regression estimates that result from fitting the quadratic model to the data.
> boot.fn=function(data,index)
+   coefficients(lm(mpg~horsepower+I(horsepower^2),data=data,subset=index))
>
> set.seed(1)
> boot(Auto ,boot.fn ,1000)
```

ORDINARY NONPARAMETRIC BOOTSTRAP

Call:
boot(data = Auto, statistic = boot.fn, R = 1000)

Bootstrap Statistics :

	original	bias	std. error
t1*	56.900099702	3.511640e-02	2.0300222526
t2*	-0.466189630	-7.080834e-04	0.0324241984
t3*	0.001230536	2.840324e-06	0.0001172164

5.4 Bootstrap

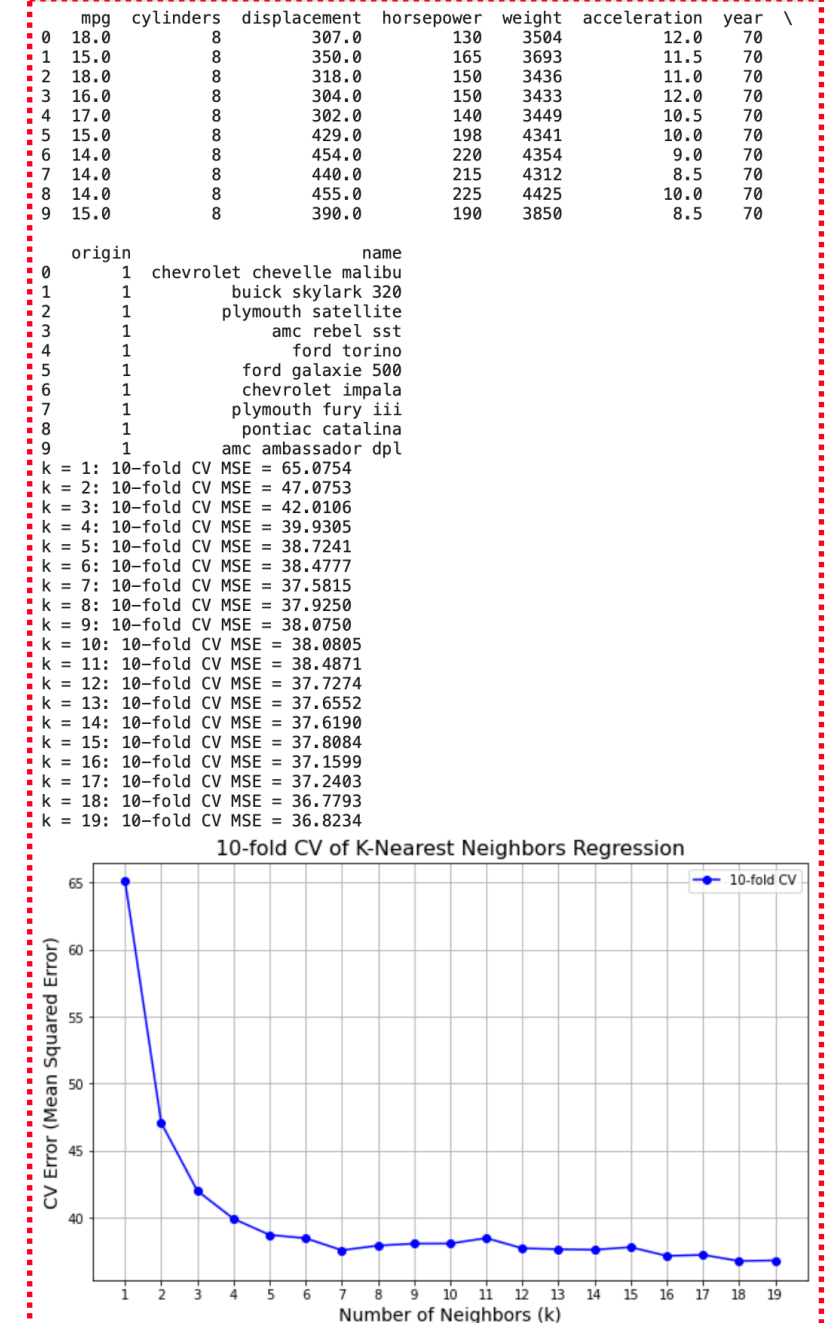
KNN regression on Auto data

```
##### Using CV to determine the best K in KNN on Auto data
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsRegressor
import matplotlib.pyplot as plt

# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset
print(Auto.head(10))
np.random.seed(1997)
x = Auto['horsepower'].values
X = pd.DataFrame({'x': x})
y = Auto['mpg']

###KNN with 10-CV
kf = KFold(n_splits=10, shuffle=True, random_state=17) # Fixed seed for reproducibility
k_values = range(1, 20)
CVERr = []
for k in k_values:
    err = 0 # Initialize error
    for train_index, test_index in kf.split(X):
        trainx, testx = X.iloc[train_index], X.iloc[test_index]
        trainy, testy = y.iloc[train_index], y.iloc[test_index]
        knnmod = KNeighborsRegressor(n_neighbors=k)
        knnmod.fit(trainx, trainy)
        test_y_pred = knnmod.predict(testx)
        squared_errors = (test_y_pred - testy) ** 2
        err += squared_errors.sum()
    cv_error = err / n_samples
    CVERr.append(cv_error)
    print(f"k = {k}: 10-fold CV MSE = {cv_error:.4f}")

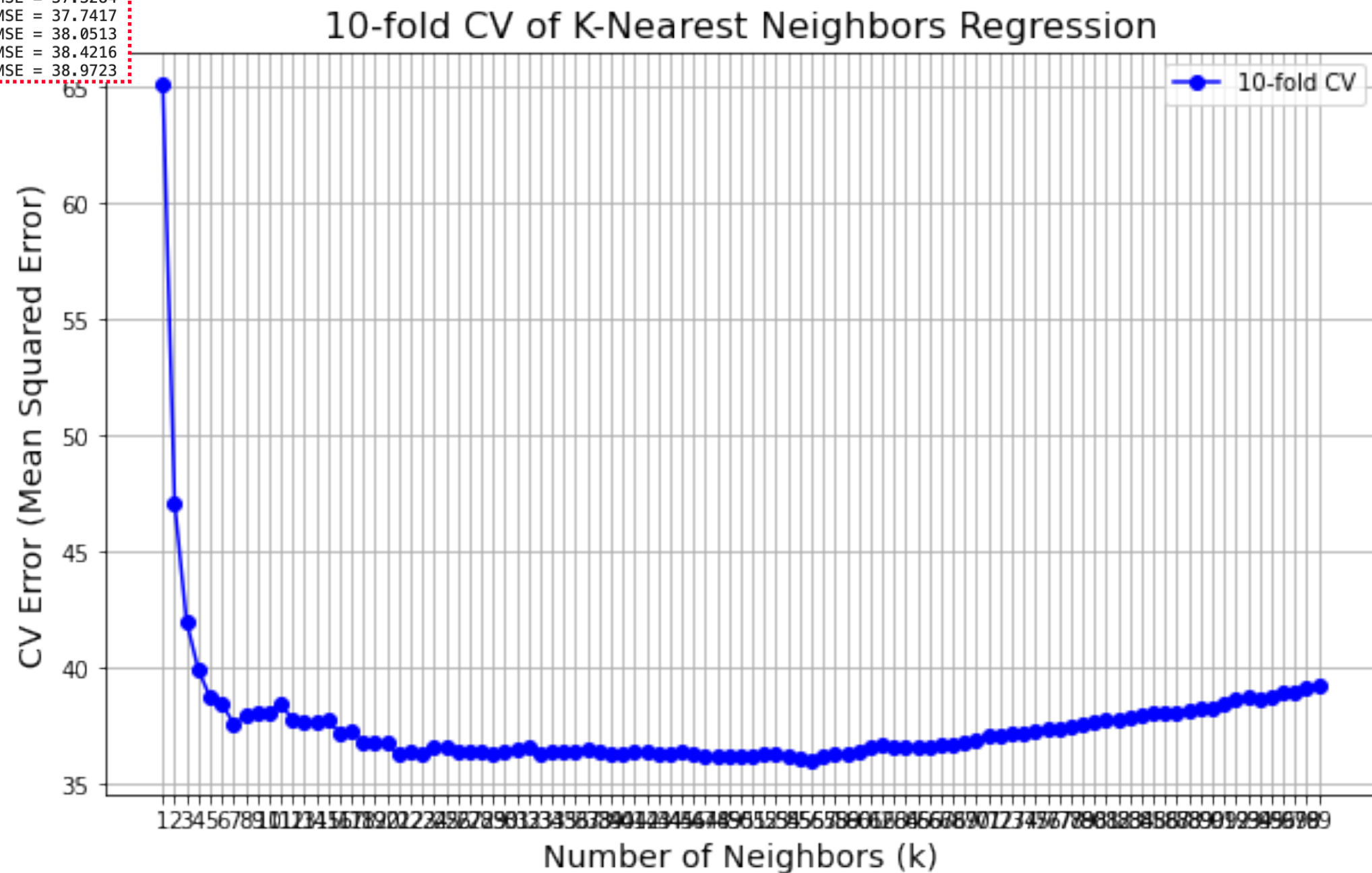
###plot
plt.figure(figsize=(10, 6))
plt.plot(k_values, CVERr, marker='o', linestyle='--',
        color='blue', label='10-fold CV')
plt.title('10-fold CV of K-Nearest Neighbors Regression', fontsize=16)
plt.xlabel('Number of Neighbors (k)', fontsize=14)
plt.ylabel('CV Error (Mean Squared Error)', fontsize=14)
plt.xticks(k_values)
plt.grid(True)
plt.legend()
plt.show()
```



5.4 Bootstrap

KNN regression on Auto data

k = 1: 10-fold CV MSE = 65.0754
 k = 6: 10-fold CV MSE = 38.4777
 k = 11: 10-fold CV MSE = 38.4871
 k = 16: 10-fold CV MSE = 37.1599
 k = 21: 10-fold CV MSE = 36.2512
 k = 26: 10-fold CV MSE = 36.3538
 k = 31: 10-fold CV MSE = 36.4645
 k = 36: 10-fold CV MSE = 36.4268
 k = 41: 10-fold CV MSE = 36.3645
 k = 46: 10-fold CV MSE = 36.2781
 k = 51: 10-fold CV MSE = 36.2311
 k = 56: 10-fold CV MSE = 36.0000
 k = 61: 10-fold CV MSE = 36.6286
 k = 66: 10-fold CV MSE = 36.6134
 k = 71: 10-fold CV MSE = 37.1062
 k = 76: 10-fold CV MSE = 37.3284
 k = 81: 10-fold CV MSE = 37.7417
 k = 86: 10-fold CV MSE = 38.0513
 k = 91: 10-fold CV MSE = 38.4216
 k = 96: 10-fold CV MSE = 38.9723



5.4 Bootstrap

KNN regression on Auto data

```
##### confidence interval of KNN regression on Auto data by Bootstrap
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.utils import resample
import matplotlib.pyplot as plt
import seaborn as sns

# Set random seed for reproducibility
# Load dataset
Auto = pd.read_csv("Auto.csv") # Replace with the actual path to the dataset
np.random.seed(1997)
x = Auto['horsepower'].values
y = Auto['mpg'].values
data = pd.DataFrame({
    'x': x,
    'y': y
})
knn=7
predict_point = pd.DataFrame({'x': np.array([155])});
knn = KNeighborsRegressor(n_neighbors=knn_k)
knn.fit(data[['x']].values, data[['y']].values)
knnpred=knn.predict(predict_point[['x']])[0]
print(f"KNN prediction at point {predict_point['x'].values[0]}: {knnpred[0]}")

n_bootstraps = 1000 # Number of bootstrap samples
knn_k = 7 # Number of neighbors for KNN
bootstrap_preds = []
for i in range(n_bootstraps):
    bootstrap_sample = resample(data, replace=True, n_samples=n_samples)
    X_boot = bootstrap_sample[['x']].values
    y_boot = bootstrap_sample[['y']].values
    knn = KNeighborsRegressor(n_neighbors=knn_k)
    knn.fit(X_boot, y_boot)
    pred = knn.predict(predict_point[['x']])[0]
    bootstrap_preds.append(pred)

bootstrap_preds = np.array(bootstrap_preds)
lower_bound = np.percentile(bootstrap_preds, 2.5)
upper_bound = np.percentile(bootstrap_preds, 97.5)
confidence_interval = (lower_bound, upper_bound)
print(f"95% Confidence Interval at point {predict_point['x'].values[0]}:")
print(f"Lower Bound: {lower_bound:.4f}")
print(f"Upper Bound: {upper_bound:.4f}")

# Plot the Bootstrap Distribution
plt.figure(figsize=(10, 6))
sns.histplot(bootstrap_preds, bins=30, kde=True, color='skyblue', edgecolor='black')
plt.axvline(lower_bound, color='red', linestyle='--', label=f'2.5th Percentile ({lower_bound:.2f})')
plt.axvline(upper_bound, color='red', linestyle='--', label=f'97.5th Percentile ({upper_bound:.2f})')
plt.title(f'Bootstrap Distribution of KNN Predictions at Point 155')
plt.xlabel('Predicted y')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

KNN prediction at point 155: 14.48
 95% Confidence Interval at point 155:
 Lower Bound: 13.4286
 Upper Bound: 15.7004

